



Universidad Politécnica
de Madrid



**Escuela Técnica Superior de
Ingenieros Informáticos**

Grado Universitario en Ciencia de Datos e Inteligencia
Artificial

Trabajo Fin de Grado

**Análisis sobre la Utilización de
Transformers y Modelos Generativos
para Generación de Anuncios.**

Autor: Guillermo Díaz Benito.

Tutor: Emilio Serrano Fernández.

Madrid, Junio, 2024.

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Grado

Grado Universitario en Ciencia de Datos e Inteligencia Artificial.

Título: Análisis sobre la Utilización de Transformers y Modelos Generativos
para Generación de Anuncios.

Junio, 2024.

Autor: Guillermo Díaz Benito

Tutor:

Emilio Serrano Fernández

Inteligencia Artificial

ETSI Informáticos

Universidad Politécnica de Madrid

Resumen

En este trabajo se desarrolla un sistema para la generación y mejora de anuncios de video mediante la utilización de modelos de inteligencia artificial generativa. La propuesta se divide en dos etapas, un primer acercamiento dónde el usuario introduce una descripción (prompt) inicial corta (sencilla, ya que está enfocado a usuarios no especializados en inteligencia artificial) que describe la idea del anuncio a generar y su objetivo (lo que trata de conseguir el usuario/empresa con el anuncio). Este prompt se refina y amplía utilizando un agente RAG (que utiliza distintos LLMs a elección del usuario implementados a través de la API de groq), adaptándolo a las características específicas del anuncio. Una vez realizado este proceso, se da la segunda etapa, donde la descripción (prompts) final (respuesta del agente) pasa por un modelo de generación de video (externo a la herramienta) para la creación final del anuncio, ofreciendo la herramienta un espacio para visualizar el resultado final.

Durante el desarrollo del trabajo, se estudian diferentes estructuras de modelos de lenguaje, buscando los mejores prompts o la identificación de un estilo óptimo en la redacción del prompt para la generación de anuncios. A su vez, se examinan los aspectos de diseño necesarios para optimizar la generación de los prompts (como la introducción de un prompt del sistema o system prompt y la evaluación de los métodos de aprendizaje de few-shot learnig y zero-shot-learning). Además, se construye una interfaz gráfica sencilla e intuitiva para acercar la herramienta a usuarios no expertos de una manera simple.

Cabe destacar la implementación de un sistema interactivo, estilo “ChatBot” con memoria, que permite al usuario validar los prompts generados o proporcionar nueva información para así construir un prompt final adecuado.

Los resultados a lo largo de la evolución del trabajo fueron demostrando mejoras significativas en la generación de texto y tiempos de respuesta, sobre todo con la eliminación de las herramientas de resumen y la unificación de la base de conocimiento del agente RAG a un único documento. Esto optimizo el sistema tanto en precisión de los textos generados en cuanto a textos esperados como en eficiencia del sistema. Aún así la herramienta tiene infinidad de mejoras empezando por la integración de modelos de video en la propia herramienta y siguiendo por la reducción de los tiempos de carga del agente RAG.

En conclusión, este enfoque proporciona una herramienta valiosa para los profesionales de marketing y publicidad que buscan optimizar el proceso de desarrollo de anuncios.

Abstract

In this work we develop a system for the generation and improvement of video advertisements using generative artificial intelligence models. The proposal is divided into two stages, a first approach where the user introduces a short initial prompt (simple, as it is focused on users not specialised in artificial intelligence) that describes the idea of the ad to be generated and its objective (what the user/company is trying to achieve with the ad). This prompt is refined and extended using a RAG agent (which uses different LLMs of the user's choice implemented through the groq API), adapting it to the specific characteristics of the advertisement. Once this process has been carried out, the second stage takes place, where the final description (prompts) (agent response) goes through a video generation model (external to the tool) for the final creation of the advertisement, with the tool offering a space to visualise the final result.

During the development of the work, different language model structures are studied, looking for the best prompts or the identification of an optimal style in the writing of the prompt for the generation of advertisements. At the same time, the design aspects necessary to optimise the generation of prompts are examined (such as the introduction of a system prompt and the evaluation of the few-shot learning and zero-shot-learning methods). In addition, a simple and intuitive graphical interface is built to bring the tool closer to non-experts in a simple way.

It is worth mentioning the implementation of an interactive system, 'ChatBot' style with memory, which allows the user to validate the prompts generated or to provide new information in order to build a suitable final prompt.

The results throughout the evolution of the work demonstrated significant improvements in text generation and response times, especially with the elimination of summarisation tools and the unification of the RAG agent's knowledge base into a single document. This optimised the system both in terms of accuracy of the generated texts in terms of expected texts and system efficiency. Even so, the tool has many improvements starting with the integration of video models in the tool itself and continuing with the reduction of the loading times of the RAG agent.

In conclusion, this approach provides a valuable tool for marketing and advertising professionals looking to optimise the ad development process.

Tabla de contenidos

1	Introducción.....	1
1.1	Contextualización del problema.....	1
1.2	Objetivos del trabajo.....	3
2	Desarrollo	4
2.1	Marco Teórico.....	4
2.1.1	Fundamentos de la Inteligencia Artificial Generativa (GenAI)	4
2.1.1.1	Transformers	4
2.1.1.2	Large Language Models (LLMs)	6
2.1.1.3	Retrieved Augmented Generation (RAG) y Agentes	9
2.1.1.4	Inteligencia Artificial Generativa para Video.....	13
2.1.2	Trabajos Relacionados	14
2.1.2.1	Inteligencia Artificial Generativa en la publicidad.....	14
2.1.2.2	Pipelines de LLMs	15
2.2	Metodología	16
2.2.1	Metodología de trabajo	16
2.2.2	Conjunto de datos.....	17
2.2.3	Entorno de ejecución	18
2.2.4	Primera Iteración	18
2.2.4.1	Preproceso	18
2.2.4.2	Modelado	19
2.2.4.3	Evaluación.....	21
2.2.5	Segunda Iteración	22
2.2.5.1	Preproceso	22
2.2.5.2	Modelado	23
2.2.5.3	Evaluación.....	24
2.2.5.4	Elaboración de la interfaz web	25
3	Resultados y Conclusiones.....	37
3.1	Discusión	37
3.1.1	Comparación con trabajos relacionados e interpretación de los resultados obtenidos	37
3.1.2	Limitaciones y posibles mejoras	38
3.1.3	Vinculación con los objetivos de desarrollo sostenible	38
3.2	Conclusiones y Futuras Líneas de Investigación.....	39
4	Bibliografía	41
5	Anexos.....	¡Error! Marcador no definido.

Índice de Ilustraciones

Figura 1: Estructura de un transformer	5
Figura 2: LLMs dentro de la IA	7
Figura 3: Mecanismos de atención	8
Figura 4: Arquitectura de un pipeline RAG básica	10
Figura 5: Tool use y ciclo de razonamiento de un agente	12
Figura 6: Ejemplo de la metodología SCRUM	16
Figura 7: Ciclo de un Sprint	16
Figura 8: Ejemplo de flujo CRISP_DM	17
Figura 9: Implementación del agente ReAct	20
Figura 10: Implementación del agente final.....	23
Figura 11: Acontecimiento 1 detallado	27
Figura 12: Acontecimiento 2 detallado	27
Figura 13: Acontecimiento 3 detallado	28
Figura 14: Acontecimiento 4 detallado	28
Figura 15: Matriz de trazabilidad	28
Figura 16: Diseño de la página de inicio	29
Figura 17: Diseño página ChatBot	29
Figura 18: Diseño de la página de visualización de videos	29
Figura 19: Diseño de la página de visualización de videos 2	30
Figura 20: Página de Inicio	31
Figura 21: Página de Inicio 2.....	32
Figura 22: Página del ChatBot	32
Figura 23: Página Visualizador de Videos	32
Figura 24: Página Visualizador de Videos 2	33
Figura 25: Página de inicio Móvil	33
Figura 26: Página de inicio Móvil 2.....	34
Figura 27: Página de ChatBot móvil	34
Figura 28: Página de video móvil	35
Figura 29: Página de video móvil 2.....	35
Figura 30: Página de video móvil 3.....	36

1 Introducción

1.1 Contextualización del problema

30 de noviembre de 2022, una fecha que a simple vista no dice nada pero que marcó un antes y un después en nuestras vidas y, concretamente en la historia de la inteligencia artificial. El 30 de noviembre de 2022, la empresa estadounidense, ahora mundialmente conocida, OpenAI lanzó al público ChatGPT (Chat Generative Pre-Trained Transformer) un bot conversacional que cautivó al mundo por su capacidad sin precedentes para generar texto con calidad humana, traducir idiomas, escribir diferentes tipos de contenido (desde contenidos muy formales a otros con carácter más creativo) e incluso responder preguntas de manera informativa.

El impacto de ChatGPT fue inmediato y trascendió el ámbito puramente tecnológico, dando a conocer el potencial de los modelos de lenguaje largo (LLMs) y la inteligencia artificial generativa (GenAI) para transformar la mayoría de los sectores del mercado, empezando por la atención al cliente, siguiendo por la educación y hasta el entretenimiento lo que despertó un gran interés y entusiasmo en todo el mundo.

Precisamente es esta capacidad para la generación de contenido completamente nuevo y original a partir de datos no estructurados lo que caracteriza a la inteligencia artificial generativa (GenAI) y en específico a los modelos de lenguaje largo (LLMs). ¿Pero qué son realmente los modelos de lenguaje largo (LLMs) y la inteligencia artificial generativa (GenAI)? La inteligencia artificial generativa (GenAI) es el campo de la inteligencia artificial centrado en el desarrollo de modelos capaces de generar contenido novedoso a partir de datos con los que ha sido entrenada, diferenciándose así de la inteligencia artificial tradicional que se centra en analizar y clasificar información existente. La inteligencia artificial generativa se basa en gran medida en los modelos de lenguaje largo (LLMs), modelos de machine learning cuya base son un tipo especial de red neuronal, los transformers. Los transformers se caracterizan principalmente por su habilidad para escalar eficientemente, lo que hace posible entrenar estos modelos con una cantidad masiva de datos de texto. En definitiva, la inteligencia artificial generativa se desarrolla en la intersección de dos campos de la inteligencia artificial profundamente estudiados, el procesamiento de lenguaje natural y el Deep Learning.

Habiendo quitado la magia que envuelve a la inteligencia artificial generativa (GenAI) y, en específico, a los modelos de lenguaje largo (LLMs), centrémonos en lo que hace a la inteligencia artificial generativa tan atractiva: la flexibilidad y creatividad. Dos características compartidas en infinidad de sectores como la educación, el diseño, la arquitectura, el entretenimiento o el tema que se aborda en este Trabajo de Fin de Grado, la publicidad y el marketing.

La industria publicitaria se encuentra en un momento de constante transformación y evolución con la necesidad de adaptarse a un panorama digital cada vez más dinámico y competitivo, donde las audiencias son cada vez más diversas y segmentadas con distintos intereses y comportamientos por lo tanto los anunciantes requieren de flexibilidad en la creación de anuncios para responder rápidamente a las tendencias emergentes y necesidades específicas de sus clientes.

Además, vivimos en una sociedad donde la sobreabundancia de información y publicidad en el mundo digital está a la orden del día por lo tanto destacar y captar la atención de las audiencias es cada vez más complicado por tanto los anuncios necesitan ser creativos, originales y atractivos para destacar entre el ruido y dejar una impresión duradera en los consumidores.

El equilibrio entre flexibilidad y creatividad es un factor determinante para tener en cuenta cuando se crea un anuncio ya que la libertad creativa puede verse limitada al adaptarse demasiado a las necesidades específicas de las audiencias y, por el otro lado un exceso de creatividad puede alejarse del mensaje central de la marca o no ser relevante para la audiencia. Es aquí donde la utilización de la GenAI surge como una nueva posibilidad para proporcionar herramientas útiles que ayuden a los anunciantes a lograr este equilibrio a la vez que optimizan el rendimiento de las campañas publicitarias.

Desde la aparición de estas nuevas tecnologías, se han desarrollado y utilizado herramientas de GenAI que cumplen con los nuevos retos que han surgido en el sector de la publicidad mostrando resultados muy prometedores tanto en anuncios de texto, pero sobre todo en anuncios que utilizan imágenes, donde incluso mejoran los generados por humanos.

Sin embargo, el éxito de estas herramientas no se limita únicamente a su implementación. Detrás de cada anuncio generado, yace un elemento crucial: un prompt de calidad. Este prompt (que podría traducirse al español como “guía detallada”) es fundamental para orientar a los modelos de inteligencia artificial generativa hacia la construcción de resultados aceptables. Aunque a simple vista pueda parecer una tarea sencilla, la creación de prompts efectivos requiere de un entendimiento profundo de los patrones y estructuras que mejoran el rendimiento del modelo.

Este conocimiento es un conocimiento específico del ámbito de la inteligencia artificial pero que es esencial a la hora de utilizar estas herramientas de inteligencia artificial generativa y obtener los mejores resultados, pero que puede ser costoso y complejo de aprender para personas no especializadas en el ámbito de la inteligencia artificial. Así, mientras que las herramientas de GenAI ofrecen un potencial sin precedentes en la creación de anuncios, también surgen retos como la accesibilidad y usabilidad de los modelos.

Por tanto, en este trabajo se pretende desarrollar una herramienta que acerque estos modelos de inteligencia artificial generativa a usuarios no especializados de manera que puedan obtener resultados cercanos a los que obtendría un experto en la materia, poniendo el foco en la generación de anuncios de video. Un ámbito muy reciente y poco explorado, teniendo como base los avances realizados en los anuncios de texto e imágenes.

Para ello, se elabora un pipeline de modelos generativos donde el usuario describirá brevemente el anuncio que quiere generar mediante un prompt corto. Este prompt corto pasará por modelo de lenguaje largo (LLM) que generará un prompt más largo y adecuado para el modelo final que generará el anuncio de video.

1.2 Objetivos del trabajo

En un mundo cada vez más digitalizado, dónde la información es el bien máspreciado, el contenido publicitario efectivo y personalizado es crucial para las empresas. La creación de anuncios que capten la atención del consumidor y le creen interés en la marca y los productos de esta, es un objetivo relevante que puede resultar costoso de conseguir de manera tradicional para las empresas.

En este contexto, la Inteligencia Artificial y los modelos de lenguaje basados en Transformers han demostrado ser herramientas útiles para la generación automatizada de contenido publicitario (tanto escrito cómo visual), mejorando el “engagement” de los anuncios generados con Inteligencia Artificial en comparación con los tradicionales generados por un grupo de expertos.

En este Trabajo de Fin de Grado se quiere investigar, diseñar e implementar un pipeline de modelos de lenguaje que simplifique el proceso de creación de anuncios, acercándolo a la utilización por personas no especializadas en el campo de la Inteligencia Artificial.

Puesto que estos modelos de lenguaje son más efectivos dependiendo de las descripciones (prompts) dadas y su estructura, se busca encontrar un modelo que cumpla características adecuadas para la generación de los mejores prompts para la generación de anuncios en video.

Para lograrlo, es necesario cumplir con una serie de objetivos específicos:

- Diseño del agente RAG para la generación de anuncios. Definición de la arquitectura e interacción entre los distintos componentes.
- Implementación del framework. Implementación e integración de los componentes del agente y modelos de lenguaje (LLMs).
- Optimización del proceso de generación de prompts.
- Evaluación de la propuesta y análisis de los resultados.
- Desarrollo de la interfaz de usuario.

2 Desarrollo

2.1 Marco Teórico

2.1.1 Fundamentos de la Inteligencia Artificial Generativa (GenAI)

2.1.1.1 Transformers

Si bien la inteligencia artificial generativa se caracteriza por generar contenido creativo nunca visto, esta no sería posible sin los transformers. Los transformers son presentados por Vaswani et al. en 2017 en el artículo “Attention Is All You Need” [17] donde describe este modelo y cómo mejora a métodos anteriores basados en Redes Neuronales Recursivas (RNNs) y Redes Neuronales Convolucionales (CNNs) gracias a su capacidad para manejar dependencias a largo plazo en secuencias de datos, revolucionando así el campo del procesamiento del lenguaje natural.

Se podría afirmar, sin mucho miedo a equivocarse, que la principal característica y, por la que son tan especiales estos modelos es la “atención”. Este mecanismo permite al modelo enfocarse en diferentes partes de la secuencia de entrada (input) de manera simultánea.

Esta atención la podemos dividir en diferentes subcategorías: “atención auto-regresiva” (self-attention), “atención cruzada” (cross-attention) y “atención multi-cabeza” (multihead attention).

La atención auto-regresiva permite que cada palabra de una secuencia de entrada (input) considere todas las demás palabras de esa secuencia al mismo tiempo, en lugar de procesar cada palabra de manera secuencial como en las RNNs. Es decir, dado un conjunto de entradas $\{x_1, x_2, \dots, x_n\}$, la atención auto-regresiva permite calcular la representación ponderada de estas entradas y lo hace mediante la siguiente fórmula:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

dónde Q = queries, K = keys y V = Values son matrices derivadas de la entrada original a través de capas de proyección lineal y, d_k es la dimensionalidad de los vectores keys.

Con esta operación el modelo puede atender distintas partes de la entrada según sea necesario utilizando la similitud de las queries y las keys. Finalmente, la salida es una combinación ponderada de los values, que captura las dependencias entre las palabras de la secuencia.

La atención cruzada (cross-attention) permite que la secuencia de salida considere todas las posiciones de la secuencia de entrada. Principalmente se usa en modelos de secuencia a secuencia (seq2seq). Un ejemplo donde se utiliza principalmente es en la traducción.

Por último, está la atención multi-cabeza (multihead attention) por la que, en lugar de tener una única atención auto-regresiva, los Transformers dividen los inputs en varias “cabezas” (heads) y calculan la atención para cada una de manera independiente lo que permite al modelo captar diferentes tipos de relaciones y patrones de datos. Para ello utiliza la siguiente fórmula:

$$MultiHead(Q, K, V) = Concat(head_1, head_2, \dots, head_h)$$

Donde cada cabeza i es el resultado de aplicar la atención autoregresiva a una proyección distinta de las secuencias de entrada (inputs):

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)W^O$$

dónde W_i^Q , W_i^K , W_i^V son matrices de proyección para la cabeza i y, W^O es una matriz de proyección final que combina la salida de todas las cabezas.

Por otro lado, es esencial describir la estructura de los Transformers donde tenemos capas de codificación (Encoder layers) y capas de decodificación (Decoder layers).

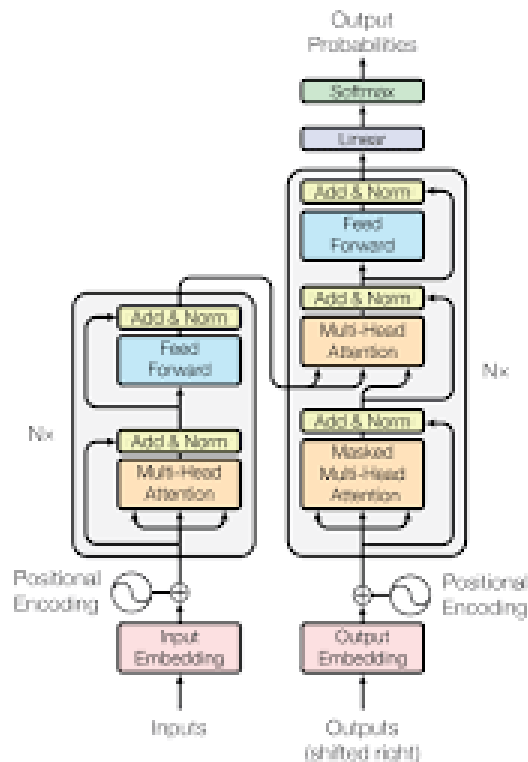


Figura 1: Estructura de un transformer (Fuente: Medium) ¹

En las capas de codificación se transforma la secuencia de entrada a una representación interna capturando las relaciones y características de esta. Además, cada capa de codificación está compuesta por: una subcapa de atención multi-cabeza y una subcapa feed-forward (que no deja de ser una red neuronal feed-forward). Cabe resaltar que ambas subcapas están envueltas en conexiones residuales y normalización.

A su vez, las capas de decodificación funcionan de manera similar a las de codificación, pero incluyendo una subcapa adicional de atención cruzada lo que permite atender tanto a la salida anterior como a la representación codificada de la entrada (otorga “memoria”).

¹ Figura recuperada de: <https://medium.com/@djm9826/transformers-ab2099ce116e>

Por tanto, una capa de decodificación está compuesta por: una subcapa de atención auto-regresiva, una subcapa de atención cruzada y una subcapa feed-forward.

Por último, habiendo conocido y entendido las características y componentes principales de un transformer se puede explicar su funcionamiento,

En primer lugar, las palabras de la secuencia de entrada se transforman en vectores de embeddings (aquí se capturan las características semánticas y sintácticas). Además, se añaden embeddings posicionales para introducir información sobre el orden de las palabras, ya que los transformers no captan el orden por si mismos debido a su arquitectura no secuencial.

En segundo lugar, los embeddings pasan a través de varias capas de codificación, donde se aplican las distintas subcapas para obtener las relaciones y dependencias complejas en los datos.

En tercer lugar, los embeddings de la secuencia de salida pasan a través de capas de decodificación.

Por último, la salida del decodificador pasa por una capa lineal y una función softmax para generar la probabilidad de cada palabra en el vocabulario.

En este trabajo, es de vital importancia conocer el funcionamiento de los transformers ya que proporcionan la base para los LLMs habilitando a su vez la creación de agentes RAG con la utilización de estos. Así las bases para la creación de nuestro agente RAG especializado en la generación de prompts específicos para modelos de IA generativa orientado a la generación de anuncios de video.

2.1.1.2 Large Language Models (LLMs)

Los LLMs son modelos de IA diseñados, en un principio, para trabajar con texto utilizando lenguaje natural (aunque se ha extendido a otros tipos de datos como las imágenes, la voz o el video más recientemente). Estos modelos utilizan técnicas de Deep Learning, en específico redes neuronales basadas en Transformers, para procesar y generar texto.

La gran diferencia entre los LLMs y los enfoques tradicionales en los campos de NLP y Deep Learning es que los LLMs utilizan grandes volúmenes de datos no etiquetados para aprender patrones lingüísticos complejos, lo que conlleva una utilización y necesidad mayor de recursos, tanto en términos de capacidad como de rendimiento. Además, los LLMs resuelven los desafíos en la captura de contextos largos y complejos que afrontaban los modelos tradicionales. Esto es un avance primordial, ya que disminuía la coherencia y relevancia de los textos generados. Por último, cabe destacar la versatilidad que proporcionan los LLMs, así como la capacidad de generalización gracias al entrenamiento con datos no etiquetados y su arquitectura. A este fenómeno se le conoce como few-shot learning (aprendizaje por unos pocos ejemplos) que les hace capaces de abordar una amplia gama de tareas lingüísticas sin necesidad de realizar fine-tuning (ajuste fino) para cada una.

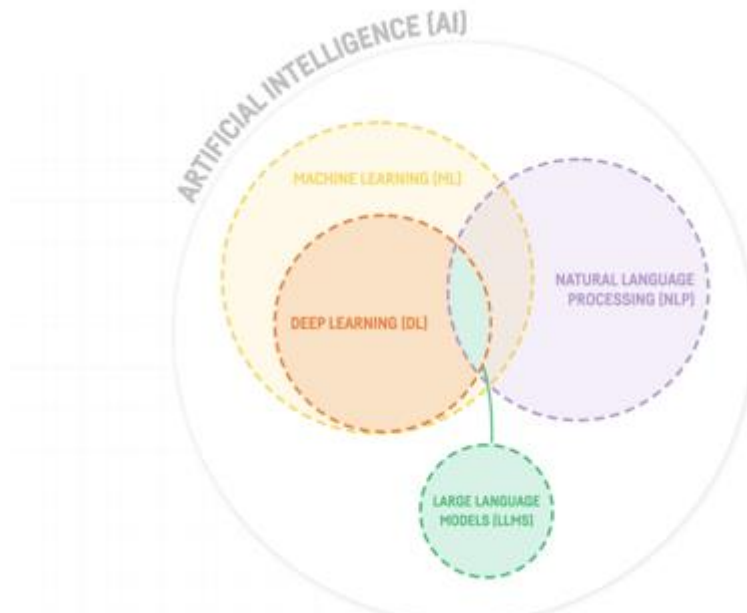


Figura 2: LLMs dentro de la IA (Fuente: médium)

Habiendo explicado detalladamente las diferencias con los modelos de Deep Learning y Procesamiento del Lenguaje Natural (NLP), es esencial entender la arquitectura de los LLMs.

Esta arquitectura esta principalmente diseñada para maximizar la capacidad del modelo para comprender y generar texto en lenguaje natural. Por tanto, un LLM está compuesto por, en primer lugar, los embeddings de entrada. Estos embedding son representaciones vectoriales de las palabras o tokens del texto de entrada. Este primer paso es vital ya que las palabras se transforman en vectores en un espacio de alta dimensión por lo que las relaciones semánticas entre palabras similares son fácilmente detectables mediante la proximidad de sus vectores. Como los LLMs utilizan transformers los embeddings necesarios son, obviamente los Word embeddings pero además son necesarios los positional embeddings por la no capacidad de captar el orden de los transformers, como se explica en el apartado anterior.

En segundo lugar, los LLMs se componen de capas de transformadores que conforman el núcleo de un LLM, donde cada capa transformer se divide en 2 subcomponentes principales: el mecanismo de atención (attention mechanism) y la red neuronal feed-forward. El mecanismo de atención a su vez se compone por la atención escalable (scaled dot-product attention) que permite al modelo centrarse en las partes relevantes del texto y la atención multi-cabeza (multihead attention) que permite obtener diferentes relaciones contextuales (como se explica en el apartado anterior).

² Figura recuperada de: <https://blog.dataiku.com/large-language-model-chatgpt>

Por otro lado, la red neuronal feed-forward es una red neuronal conectada completamente que aplica transformaciones no lineales a cada token individualmente.

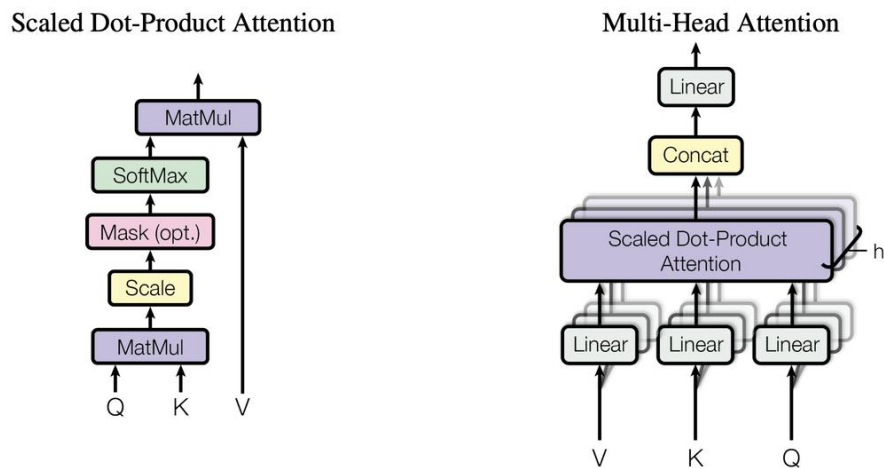


Figura 3: Mecanismos de atención (Fuente: *Attention is all you need*)

En tercer lugar, los LLMs al igual que los transformers, incluyen capas de normalización y conexiones residuales que mejoran la estabilidad del entrenamiento y eficiencia del modelo.

Las conexiones residuales (residual connections) ayudan a mitigar el problema del desvanecimiento del gradiente permitiendo que el flujo de gradientes pase a lo largo de la red y por tanto facilitando el entrenamiento de las redes neuronales profundas (DNN), mientras que la normalización de las capas (layer normalization) se utilizan para acelerar el entrenamiento normalizando los valores de activación de cada capa.

Por último, las representaciones internas del modelo se transforman para que puedan ser usadas, normalmente en predicciones de palabras. Para ello se proyectan las representaciones lineales sobre el espacio del vocabulario del modelo a través de una capa lineal y utilizando una capa softmax se convierten estas proyecciones en probabilidades, lo que permite seleccionar la palabra más probable y utilizarla como salida del modelo.

Finalmente, como se comenta al inicio de este subapartado, una característica principal de los LLMs es la utilización de corpus masivos de datos de texto no etiquetado para su entrenamiento. Este entrenamiento se compone de dos fases: preentrenamiento y fine-tuning (ajuste fino).

En la etapa de preentrenamiento se utiliza este corpus masivo de datos no etiquetados (libros, artículos, contenido web, etc.) con el objetivo de predecir la siguiente palabra en una secuencia, permitiendo al modelo capturar patrones generales del lenguaje, adquiriendo conocimientos sobre distintos temas y estilos de escritura durante este proceso. En el sentido más técnico el objetivo principal de esta etapa es minimizar la pérdida en la predicción de la siguiente palabra (utilizando una función de pérdida como “cross entropy”).

El fine-tuning se utiliza para “especializar” el modelo a ciertas aplicaciones, mejorando su precisión y rendimiento en esas tareas. Para ello utiliza datos etiquetados específicos de las aplicaciones a especializar y ajusta los parámetros del modelo para minimizar la pérdida en la tarea específica. Cabe resaltar que esta etapa es opcional en el entrenamiento de un LLM.

En este trabajo, se utilizarán y compararán diversos LLMs ya preentrenados para construir el agente RAG especializado en la generación de prompts text-to-video orientados a la generación de anuncios. Por ello, es esencial entender su funcionamiento y estructura ya que son el pilar fundamental que sustenta el resto del trabajo.

2.1.1.3 Retrieved Augmented Generation (RAG) y Agentes

El mundo de la inteligencia artificial generativa (GenAI) está en constante desarrollo y evolución, al ser un campo muy nuevo, los avances y nuevos descubrimientos aparecen cada vez más rápido. Si bien, se comentaba en el apartado anterior la técnica de entrenamiento de LLMs del fine-tuning, surge como respuesta otra técnica basada en la combinación de técnicas de recuperación de información y generación de texto llamada Retrieval Augmented Generation (RAG).

RAG es una metodología que permite integrar datos recuperados de grandes bases de conocimiento dejando a un lado la confianza total en el modelo y aportando un contexto más enriquecedor y respuestas más precisas y coherentes en el ámbito de la base de conocimiento. Por lo tanto, el modelo RAG será muy útil para tareas que requieran de un conocimiento específico y detallado.

En comparación con fine-tuning, RAG es menos eficiente que el fine-tuning y además es más costoso de implementar ya que requiere de la integración de un retriever (recuperador) y un generator (generador) pero a su vez para tareas que requieren respuestas precisas y basadas en hechos es más adecuado. Por otro lado, RAG no depende de la calidad, ni de un dataset etiquetado, lo que lo hace más accesible.

RAG, como se comentaba al inicio de la sección, se basa en dos principios o componentes básicos: el retriever (recuperador de información) y el generator (generador de texto).

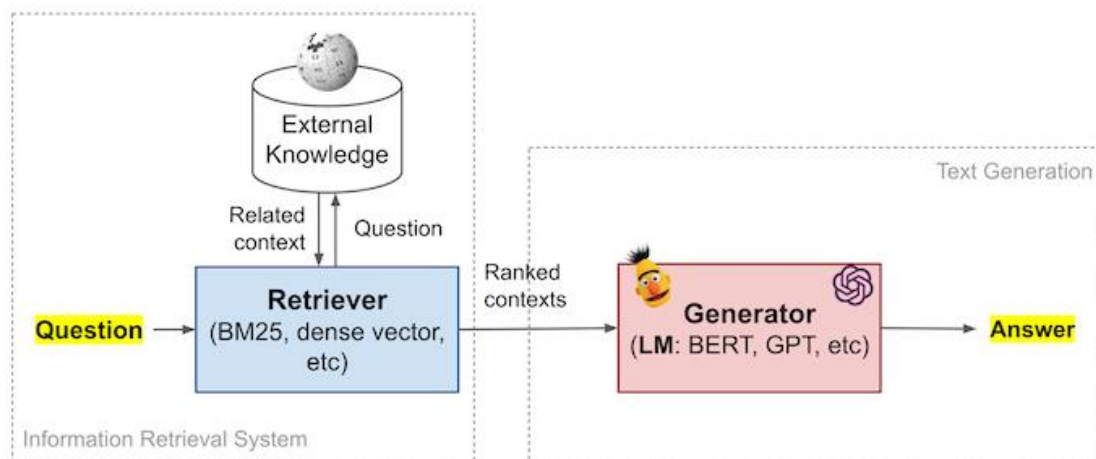


Figura 4: Arquitectura de un pipeline RAG básica (Fuente: Medium)³

El retriever como su propio nombre indica utiliza técnicas avanzadas de recuperación de información para buscar y seleccionar documentos o secciones relevantes dentro de un documento de una base de conocimiento (por lo tanto, una necesidad de tener una ya sea representada de manera tradicional o vectorial).

Los modelos de recuperación mayormente utilizados son:

- DPR (Dense Passage Retrieval) el cual descentraliza como se guarda el conocimiento dentro de una red neuronal profunda (DNN) creando múltiples caminos de acceso a la misma información. DPR entrena dos redes de BERT, una para consultas y otra para pasajes (secciones de documentos). Estas crean embeddings vectoriales los cuales permiten realizar una búsqueda basada en similitudes.
- BERT (Bidirectional Encoder Representations from Transformers) models son una familia de modelos como BERT o sus distintas variantes (como RoBERTa o ALBERT) utilizados para generar embeddings de consultas y documentos, los cuales se utilizan para calcular distintas métricas de distancia (como la similitud del coseno).

Después de este preproceso y utilizando los modelos de recuperación, los documentos son convertidos en vectores y almacenados en un índice vectorial (espacio de alta dimensión), el cual permite búsquedas rápidas utilizando técnicas como LSH (Locality Sensitive Hashing) por la que se construye el índice usando una función hash.

Por tanto, el funcionamiento de un retriever es sencillo: la consulta de entrada se convierte en un vector mediante un modelo de recuperación, se busca el índice vectorial utilizando el vector de la consulta para encontrar documentos o pasajes similares y, por último, se recuperan los documentos o pasajes basándose en la similitud de los vectores.

³ Fuente recuperada de: <https://medium.com/@daniel.fleischer/open-domain-q-a-using-dense-retrievers-in-fastrag-65f60e7e9d1e>

El generator por otro lado, podríamos decir que depende del retriever, ya que utiliza los documentos o pasajes recuperados para generar las respuestas basándose en modelos generativos avanzados de NPL.

Un generator se compone de un modelo generativo, ya sea BART (Bidirectional and Auto-Regressive Transformers) que combina un preentrenamiento bidireccional con uno autoregresivo para generar respuestas de calidad o T5 (Text-To-Text Transfer Transformer) que convierte las tareas de procesamiento del lenguaje en problemas E/S (entrada-salida) de texto, lo que permite una gran flexibilidad y capacidad de generación.

Además del modelo generativo un generator también se compone de mecanismos de atención para enfocar las partes relevantes del contexto (documentos recuperados por el retriever) al generar cada token de respuesta.

Entonces un generator incorpora los documentos recuperados al modelo como parte del contexto que el modelo utilizará para generar una respuesta token por token (el mecanismo de atención ayudará a enfocarse en las partes del contexto relevantes) y, por último, se utilizan técnicas como beam search (para explorar diferentes secuencias de salida y seleccionar la mejor) para mejorar la coherencia y fluidez de las respuestas.

Asentadas las bases de un modelo RAG, veamos cómo ha evolucionado y en qué dirección esta metodología.

Los agentes RAG representan a la perfección esta evolución permitiendo manejar consultas complejas y proporcionar respuestas precisas basadas en volúmenes de datos masivos. Los agentes RAG a diferencia del pipeline RAG estándar combinan las técnicas de recuperación de información con la síntesis de texto de los LLMs para conseguir estos avances.

Los agentes RAG son estructuras complejas construidas a partir de varios componentes esenciales, por tanto, para la comprensión adecuada de estos agentes es necesario profundizar entendiendo sus componentes principales.

En primer lugar, debemos destacar los índices, estructuras optimizadas para las tareas de recuperación y generación de información de datos que contienen referencias a estos datos almacenados, incluyendo metadatos que describen las características y el contenido de los datos indexados. La inclusión de estos metadatos es de vital importancia, ya que permitirá a los query engines realizar funciones de filtrado y de priorización lo que mejorará las respuestas.

En segundo lugar, cómo se ha introducido en el párrafo anterior, tenemos los query engines o motores de consulta. Estos actúan como interfaz de búsqueda sobre los datos de un índice, respondiendo a tipos específicos de preguntas. Es decir, cada query engine está especializado en una tarea en específico y se deberá elegir un motor u otro dependiendo de la consulta de entrada.

Por último, tenemos las query tools o herramientas de consulta, que podríamos definir como una abstracción de los motores de consulta que incluyen metadatos con descripciones del tipo de pregunta que puede responder, lo que permite una recuperación mucho más precisa y contextualmente relevante.

Una vez desglosadas las componentes principales de un agente RAG, entendamos las diversas formas de implementarlo, desde la más simple a la más compleja.

La forma más simple de agente RAG son los llamados “Routers”. Un router ante una query de entrada, selecciona uno entre varios query engines (motores de consulta) para ejecutar la búsqueda. Esta capacidad para entender dinámicamente la consulta es un avance diferencial con el RAG tradicional. Un router analiza la query de entrada para determinar qué query engine (motor de búsqueda) es el más adecuado para la tarea a realizar, escoge dinámicamente entre los query engines dependiendo el tipo de pregunta y finalmente realiza la búsqueda usando el query engine seleccionado combinando la recuperación de información con el LLM.

Una forma un poco más avanzada serían agentes que utilizan tool calling, la cual permite a los LLMs interactuar con las herramientas y sistemas externos de manera dinámica. El tool calling añade una capa de entendimiento de queries en el pipeline RAG haciendo posible que los LLMs no se utilicen únicamente para la síntesis de información, sino que puedan inferir los argumentos necesarios para la ejecución de cada herramienta, proporcionando contextos específicos con parámetros detallados y así poder responder a queries más complejas con resultados más precisos. Además, permite trabajar con filtros de metadatos para cada herramienta lo que mejora aún más la precisión en las respuestas.

En tercer lugar, debemos destacar los agentes con un ciclo de razonamiento completo. Este ciclo de razonamiento permite al agente razonar sobre múltiples herramientas y pasos utilizando tool calling y, está compuesto por un Agent Worker el cual se encarga del razonamiento sobre tareas y el entorno (herramientas y LLMs) y por un Agent Runner que actúa como el orquestador de tareas del agente y que a su vez esta compuesto por un Agent State, que mapea el task_id a un task state (incluyendo tarea, pasos completados y cola de pasos a realizar) y, por una memoria de conversación donde se almacena el contexto de las interacciones (proporcionando una referencia continua).



Figura 5: Tool use y ciclo de razonamiento de un agente (Fuente: Jerry liu)

Este tipo de agentes proporciona una serie de beneficios respecto al resto como, un desacoplamiento de la creación y ejecución de tareas mejorando la eficiencia operativa dada la flexibilidad en la ejecución de las tareas o una depuración (debuggability) y direccionalidad (steerability) mejoradas ofreciendo experiencias de usuario que permiten la modificación de respuestas en pasos intermedios lo que incorpora una retroalimentación humana proporcionando un control más refinado en la generación de respuestas.

Para finalizar, describiremos brevemente la evolución de este último tipo de agente RAG (que será el que utilicemos en este trabajo, ya que, solo utilizaremos un documento) que son los agentes multi-documentos. Estos siguen en la línea de estos últimos agentes, diferenciándose en que en este caso específico el agente realiza “retrieval augmentation” al nivel de herramientas y no al nivel de texto, ya que al tener un gran número de herramientas el LLM puede fallar a la hora de escoger la herramienta (pueden no haber todas las herramientas en el prompt aumentando rápidamente y de forma considerable los costes y la latencia porque se está aumentando el número de tokens).

2.1.1.4 Inteligencia Artificial Generativa para Video

Los modelos de generación de video o más concretamente modelos text-to-video, a lo que nos vamos a referir como modelos de video tienen como objetivo principal generar contenido de video a partir de descripciones de texto. Estos modelos además de técnicas de procesamiento de lenguaje natural (NLP) y Deep Learning, utilizan técnicas de visión por ordenador lo que les hace una tecnología muy poderosa, pero al ser muy nueva, con mucho margen de mejora también.

Estos modelos y la evolución de estos están directamente relacionados con los avances en modelos generativos (GANs, VAEs, etc.) con especial atención en los modelos de lenguaje (LLMs) los cuales se combinan para crear herramientas capaces de interpretar descripciones textuales y traducirlas a secuencias de video coherentes. Cabe destacar que dentro de esta evolución muchas de las técnicas utilizadas para modelos text-to-image han sido extrapoladas a los modelos de video.

Las principales técnicas (enfoques) para la generación de video a partir de texto incluyen: modelos basados en GANs que utilizan una estructura generator-discriminator, modelos basados en Transformers que utilizan la atención para generar cuadros de video manteniendo una estrecha relación con el texto y con los cuadros cercanos y, modelos híbridos que mezclan las ventajas de las dos propuestas anteriores.

En este trabajo nos centraremos en los modelos de pika.ai, una plataforma especializada en la generación de videos a partir de texto. Los modelos de pika destacan por distintas características diferenciadoras de otros enfoques en el ámbito de generación text-to-video. Pika utiliza una combinación de modelos multimodales que no solamente comprenden el contexto lingüístico del input de texto, sino que también entiende las complejidades visuales necesarias para generar videos realistas. A su vez, utiliza modelos híbridos combinando GANs y Transformers para garantizar que la salida de video sea coherente con la descripción textual manteniendo calidad de video.

En conclusión, la generación de videos a partir de texto es de lo más novedoso en el ámbito de la inteligencia artificial generativa y que está evolucionando gracias a arquitecturas avanzadas como las GANs y los Transformers. En este ámbito pika.ai destaca proporcionando una herramienta novedosa gracias a su enfoque multimodal lo que proporciona videos de calidad.

2.1.2 Trabajos Relacionados

2.1.2.1 Inteligencia Artificial Generativa en la publicidad

En los últimos años ha habido un creciente interés en el uso de GenAI en el sector de la publicidad, desarrollándose herramientas que ayudan a mejorar la eficacia del proceso de creación de anuncios. Los campos en los que se ha enfocado mayormente la utilización de la GenAI en la publicidad [4] son: para el proceso de construcción de una campaña digital de marketing, el diseño de anuncios, la automatización de generación de anuncios y la personalización de anuncios.

Al ser, la inteligencia artificial generativa, una tecnología reciente y cuyo interés ha surgido con mayor fervor en los últimos dos años, combinado con un sector específico como es la publicidad, todo ello hace que existan pocos trabajos relacionados que combinen ambos temas. Uno de los trabajos más interesantes [1] que relaciona la inteligencia artificial, los modelos generativos y la publicidad es el desarrollado por la universidad de Gyeongsan (República de Korea) en colaboración con el departamento de computer science de Islamia University of Bahawalpur (Pakistán) y la universidad de Chongqing (China) donde se recogen conceptos esenciales e investigaciones influyentes que han tenido un impacto significativo en el sector publicitario, destacando a su vez la importancia de la utilización de inteligencia artificial (en específico de modelos generativos) en el sector publicitario.

Existen diversos trabajos [5, 7] que, al igual que el anterior, respaldan el uso de la inteligencia artificial generativa como herramienta clave en el sector de la publicidad y el marketing, señalando la necesidad de la creación de documentación clara y completa para el uso correcto de las herramientas y avisando de que la integración de estas requiere inversión y formación de empleados. Sin embargo, en [2] los autores advierten que la creación de contenidos por modelos de GenAI en redes sociales reduce la percepción de autenticidad de la marca provocando el efecto contrario al intencionado, ya que la confianza de los consumidores disminuye. Pero señalan que cuando la GenAI asiste en la creación de contenidos en lugar de automatizarla, la preocupación de los seguidores disminuye.

Por otro lado, la generación de contenido personalizado es un requisito esencial para cumplir hoy en día por las marcas. Esto ha motivado la investigación en este ámbito dando resultados muy favorables mostrados en diversos trabajos [3, 6] donde se resalta que la utilización de modelos generativos para la creación de anuncios personalizados mejora los creados por humanos. Además, en [8] se propone una automatización de este proceso obteniendo resultados exitosos en busca de una automatización completa.

2.1.2.2 Pipelines de LLMs

Uno de los proyectos más destacables en cuanto a la construcción de pipelines y entornos de LLMs es el presentado por el laboratorio Mphasis [10], dónde se resaltan las plataformas, frameworks y herramientas más comunes en el proceso de creación de pipelines con LLMs. Es interesante la aportación ya que permite una generación de entornos de trabajo que generan mejores prompts, con contextos más grandes y de inferencia más rápida.

Otro artículo [15], utiliza directamente LLMs como ChatGPT-4 y Gemini en la implementación de un pipeline para generar resúmenes personalizados de calidad. Dentro del pipeline se utilizan estos dos LLM de diferente manera, en un primer instante para extraer la tarea a realizar propuesta mientras que en un segundo lugar se utilizan para crear el resumen.

A su vez, en [9] podemos ver como la concatenación de LLMs puede mejorar el rendimiento de los modelos por separado, en este caso lo hace para el reconocimiento de emociones en la voz.

Por otro lado, la utilización de pipelines [11, 12], para acelerar procesos dentro del mundo de la inteligencia artificial es algo que se busca con asiduidad, más en específico, se busca escalar y paralelizar modelos de Deep Learning o más recientemente los LLMs, consiguiendo acelerar la eficiencia de estos modelos incluso en dispositivos heterogéneos. Demostrando así la capacidad de los pipelines para conseguir resultados más eficientes.

Por último, a la hora de trabajar con pipelines de LLM se han desarrollado herramientas útiles que facilitan la gestión de estos modelos, obteniendo buenos resultados cuando se implementan en pipelines que utilizan RAG (Retrieval Augmented Generation). Algunas de estas son:

- PromptNode [13]: Nodo customizable que permite la utilización de LLMs directamente en un pipeline.
- Ranker [14]: Ordena documentos según un criterio dado. Destacan los feature-based rankers (más simples) como RecentnessRanker.
- Retriever [16]: Filtra y selecciona los documentos dentro de un DocumentStore según la consulta. Normalmente se combina con un Reader dentro del pipeline para acelerar la búsqueda.

2.2 Metodología

2.2.1 Metodología de trabajo

Para desarrollar este trabajo se ha elegido aplicar un marco de producción y desarrollo iterativo e incremental, siguiendo una metodología de tipo SCRUM, la cual es una metodología ágil que enfatiza la flexibilidad y entrega incremental de productos. SCRUM se estructura en ciclos de trabajo cortos y repetitivos llamados sprints, estos se repetirán iterativamente realizando entregas regulares del producto final al terminar estos sprints. Durante los sprints se espera realizar un conjunto de mejoras e introducción de extensiones del proyecto teniendo como punto de partida la versión final del sprint anterior.

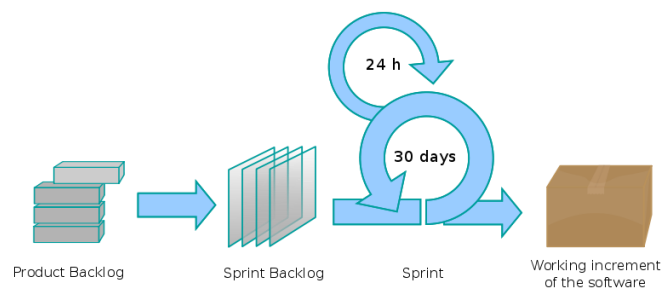


Figura 6: Ejemplo de la metodología SCRUM (fuente: Javier Garzas⁴)

Se ha elegido esta metodología frente a otras debido principalmente a que proporciona una flexibilidad y adaptabilidad única lo que la hace especialmente útil para la consecución de este trabajo ya que no existe una solución clara del producto final (se esperan cambios en los requisitos y objetivos).

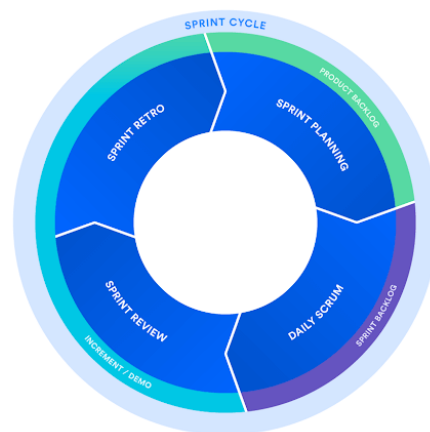


Figura 7: Ciclo de un Sprint (fuente: Atlassian)⁵

⁴ Figura recuperada de: <https://www.javiergarzas.com/2012/08/diagrama-scrum.html>

⁵ Figura recuperada de: <https://www.atlassian.com/es/agile/scrum>

Durante el primer subsprint se desarrollará la parte correspondiente al Agente RAG (teniendo en cuenta preprocesado y generación de documentos, selección de embeddings y LLMs e implementación del agente). Se ha elegido seguir una metodología CRISP_DM aceptada ampliamente en proyectos de minería de datos pero que se adaptará a IA generativa.

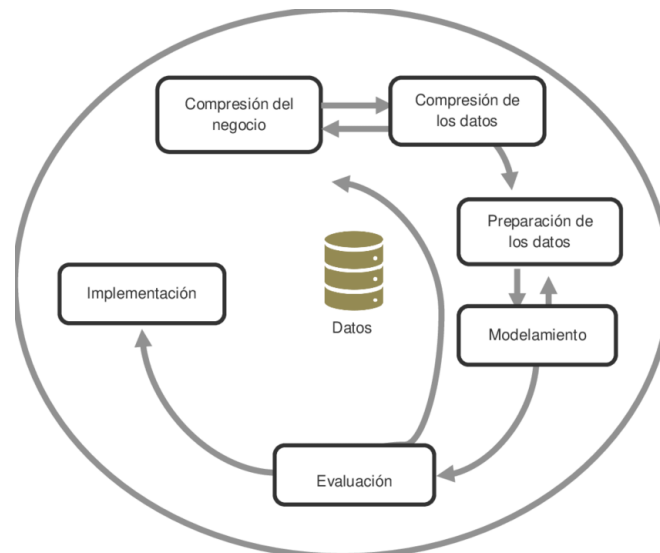


Figura 8: Ejemplo de flujo CRISP_DM (fuente: ResearchGate)⁶

En el segundo subsprint se diseñará y desarrollará la interfaz web que permitirá hacer uso del agente, además de visualizar los resultados. Esta seguirá las fases principales del desarrollo del software (análisis, diseño, desarrollo iterativo y pruebas) dotando a todo el proceso de flexibilidad (aunque ya un poco más rígida porque no se presuponen cambios drásticos).

2.2.2 Conjunto de datos

Para el desarrollo de este trabajo se van a utilizar tres fuentes distintas de datos: dos datasets específicos de anuncios de texto y uno específico de prompts text-to-video.

Dentro de los relativos a anuncios de texto se ha optado por un dataset de 1000 ejemplos de anuncios de texto creativos (con sus respectivos tamaños) y por otro lado un dataset de 2000 transcripciones de anuncios de texto reales (además incluye la categoría del anuncio, el anunciante y el producto anunciado).

Por otro lado, el dataset relacionado con prompts text-to-video contiene 1.67 millones de ejemplos de prompts reales utilizados en cuatro “diffusion models” distintos. Debido al gran tamaño de este dataset, se ha reducido el tamaño a uno similar al de los anuncios de texto.

⁶ Fuente recuperada de: https://www.researchgate.net/figure/El-proceso-de-la-Metodologia-CRISP-DM-IBM-2020_fig3_347529684

2.2.3 Entorno de ejecución

En este trabajo se ha utilizado una maquina con CPU Intel(R) Core(TM) i5-10210 CPU @ 1.60GHz 2.11 GHz, 8GB de RAM y GPU NVIDIA GeForce GTX 1060 con 6GB de RAM GDDR5.

El proyecto se ha desarrollado utilizando una versión de Python, que a su vez ha hecho uso de las librerías llama-index, llama-index-embeddings-huggingface, llama-index-llms-groq, sentence-transformers, Transformers para la implementación del Agente RAG y la librería streamlit para el desarrollo de la interfaz web, además de github y streamlit cloud como plataformas para la publicación de la interfaz web en la nube.

En cuanto al entorno de desarrollo se ha utilizado la versión estándar y gratuita de Google Colaboratory que pone a disposición del usuario un entorno de T4 GPU crucial para desarrollar el trabajo con LLMs y modelos de video, ya que permiten realizar operaciones con matrices grandes y operaciones paralelizables lo que se traduce en mejores tiempos de latencia. Además, se ha utilizado ollama y su respectiva librería en Python para probar localmente los mejores modelos.

Por lo que se refiere al desarrollo de la interfaz web se utilizó un enviroment de anaconda para el despliegue de la librería streamlit y en consecuencia la interfaz web.

2.2.4 Primera Iteración

2.2.4.1 Preproceso

En esta primera iteración se pretende generar un agente RAG mono documento con sus respectivas herramientas (summary tool and vector tool) para hacer un análisis del comportamiento y rendimiento de este. Por otro lado, se desea elaborar un primer prototipo de interfaz web que integre al agente.

Tomando como punto de partida lo explicado en el apartado de agentes RAG del marco teórico. En ese apartado se explicaba que los agentes RAG trabajan sobre una base de conocimiento, este elemento es esencial ya que es lo que le da conocimiento “experto” al agente (es el pilar sobre el que el agente construye sus respuestas).

Las bases de conocimiento pueden ser bases de conocimiento estructuradas, como bases de datos relacionales y tablas, las cuales mejoran la precisión en la recuperación de información. También se pueden dar bases de conocimiento no estructuradas, compuestas por textos libres, documentos y otros formatos no sistematizados, que utilizan técnicas de minería de textos e indexación para trabajar con este tipo de datos. Por último, están las bases de conocimiento compuestas por ontologías y grafos, las cuales ofrecen una definición explícita de las relaciones entre conceptos, facilitando la recuperación de información contextual y la integración de diversas fuentes de datos.

En este trabajo, se ha optado por construir una base de conocimiento del segundo tipo. Particularmente en esta primera iteración será una base de conocimiento compuesta por un único documento que se construirá a partir de los dos *datasets* relativos a anuncios de texto.

Se ha decidido no realizar ningún tipo de limpieza de datos dentro del *dataset*, ya que se considera que es una fuente completa y sin errores significativos para completar la base de conocimiento.

Por tanto, en el script *ads.py* se realizan las transformaciones necesarias para unificar en una serie las descripciones de los anuncios de cada *dataset*. Además, una vez unificado se ha añadido una primera línea en la que se describe el contenido del documento. Esta línea de texto, que parece ser insignificante, es de vital importancia ya que al ser una lista de ejemplos de anuncios es necesario dar contexto al agente sobre el resto del documento. Pongamos que el agente recibe una query del estilo “*genera un anuncio...*”, la herramienta que el agente que utilizará será una de las relativas a las generadas utilizando el documento *ads.txt*. Si no existiera esta frase en el documento el agente fallaría en ciertas ocasiones y no utilizaría una herramienta de las generadas utilizando el documento *ads.txt*, generando una respuesta no deseada.

Finalmente, se convierte la serie completa en un archivo de texto que será el que forme la base de conocimiento del agente.

2.2.4.2 Modelado

En esta primera iteración se va a evaluar el rendimiento del agente RAG para generar prompts largos orientados a la generación de anuncios de texto y la precisión de estos.

Para la consecución del objetivo, es crucial evaluar el LLM que va a utilizar el agente, ya que, como se detalla en el marco teórico, estos son la base del buen funcionamiento del agente. En este análisis se ha tenido en cuenta tanto el rendimiento y calidad de los resultados generados por los distintos modelos, como su viabilidad de implementación dentro de nuestro agente RAG (open source, estructura de metadatos correcta, etc.).

Los modelos que se han evaluado son: *Mistral* (en concreto el modelo *mistral-7b*), *Llama2*, *Llama3* y *Gemma*. Estos modelos se han evaluado teórica y prácticamente usando Ollama que permite la implementación en local de estos.

La evaluación del modelo elegido se explicará con mayor detalle en la próxima sección, pero finalmente se eligió el modelo *Zephyr-7b* dado que es un modelo open-source disponible en Hugging Face sin necesidad de suscripción o token de autenticación, lo que ofrece una disponibilidad total, tanto en local con Ollama (como el resto de los modelos) como en la nube (Google Colaboratory) mediante Hugging Face. Además, *Zephyr-7b* al ser un modelo derivado de un modelo *Mistral-7b* entrenado con fine-tuning para actuar como un asistente virtual ofrece un rendimiento robusto y eficiente. Por último, su estructura de metadatos es adecuada con la estructura de nuestro agente lo que le hace totalmente compatible.

Una vez seleccionado el modelo, otro factor clave a tener en cuenta en cuenta dentro de la arquitectura del agente RAG es el embedding a seleccionar. En este caso no se ha realizado una evaluación exhaustiva del embedding a utilizar y se ha elegido el embedding *BAAI/bge-small-en-v1.5* ya que ofrece una representación vectorial eficiente y está optimizado para integrarse a la perfección en modelos como *Zephyr-7b* facilitando la tarea de recuperación de información ya que ofrecen una alta precisión en la captura de relaciones semánticas entre palabras y frases. Por último, debemos destacar que gracias a su tamaño compacto (*small*) permite una utilización de recursos menor, lo que le hace perfecto para su implementación en sistemas locales.

Por tanto, habiendo asentado los pilares fundamentales sobre los que se soporta el agente RAG pasamos a construir los componentes principales del mismo.

El agente RAG tiene como componentes principales: índices vectoriales, query engines (motores de búsqueda) y query tools (herramientas), además del LLM, el agent worker y el agent runner.

Para la creación de los índices se cargan los documentos (en este caso uno), se trocean con un tamaño de 1024 tokens por nodo, la elección de este tamaño para cada nodo es porque 1024 tokens suelen ser suficientes para capturar contextos completos y a su vez no es un tamaño muy grande que exceda el context window del LLM y por tanto evita que haya pérdida de información. Finalmente se crean los índices tanto vectorial (vector index) como de resumen (summary index).

Una vez creados los índices, se crean tanto los query engines (vector query engine y summary query engine) los dos sin utilizar ningún tipo de filtro ya que el agente RAG no requiere de un filtrado en específico para la generación de las respuestas deseadas. Cabe destacar que el summary query engine utilizará todos los nodos para dar una respuesta por lo tanto se ejecutará de manera asíncrona mientras que el vector query engine no.

Por último, una vez creados los motores de búsqueda, se crean las herramientas introduciendo las descripciones específicas para cada herramienta donde se especifica que la herramienta de resumen se debe de utilizar para resumir preguntas o queries relacionadas con anuncios, mientras que en la herramienta vectorial se especifica que es útil para responder preguntas sobre anuncios.

Finalmente integramos en el agent Worker las herramientas y el LLM. Como se describe en el marco teórico, el agent worker se encargará de controlar la ejecución de las tareas paso a paso, mientras que el agent Runner llamará al Agent Worker y será el encargado de ejecutar cada tarea. Además, este último tendrá un registro del estado del agente, guardando así una “memoria de conversación”. Esto se realiza mediante la implementación de un agente ReAct (debido a las características del LLM) el cual proporciona un framework superior.

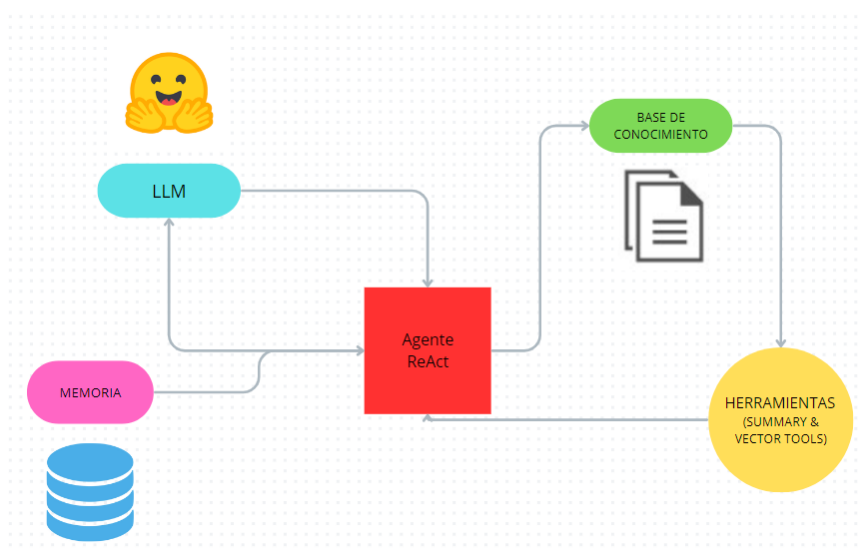


Figura 9: Implementación del agente ReAct (fuente: Creación propia)

2.2.4.3 Evaluación

Cómo se ha comentado en el apartado anterior la evaluación realizada se puede dividir en dos secciones: evaluación del LLM (anterior a la construcción del agente RAG) y evaluación del agente RAG (posterior a la construcción del agente RAG).

Como hemos destacado en la sección anterior, el modelo seleccionado en este primer sprint es el Zephyr-7b mostrando las razones de su selección, pero cuáles ha sido los contras del resto de modelos para su no selección.

Los modelos mistral son una familia de modelos open-source diseñados para tareas generales de procesamiento de lenguaje natural (NLP). En concreto el mistral-7b es un modelo equilibrado en eficiencia y rendimiento por lo que es adecuado para equipos con recursos computacionales limitados pero su disponibilidad es limitada por lo que para proyectos como este que requiere flexibilidad es menos competitivo.

Por otro lado, tenemos el modelo Llama2 el cual fue desarrollado por Meta. Este modelo mejora en precisión y eficiencia a su modelo anterior proponiendo un uso eficiente de recursos, pero no es completamente open-source lo que complica su implementación e integración en nuestro sistema, además existe una evolución de este modelo que lo mejora en precisión y eficiencia.

El modelo Llama3 es el modelo más avanzado de Meta, pero arrastra las deficiencias en disponibilidad de su modelo anterior y en comparación con Zephyr-7b es un modelo más rígido para la personalización.

Por último, tenemos el modelo Gemma el cual tiene capacidades de comprensión y generación de lenguaje natural (NLP) muy sofisticadas lo que le hace muy versátil en aplicaciones NLP, pero es muy costoso de implementar lo que lo hace imposible de utilizar en proyectos de pocos recursos como este.

Una vez descrita la evaluación de LLMs pasaremos a mostrar la evaluación del rendimiento de nuestro agente con el embedding y el LLM seleccionado en este sprint.

Se ha elegido utilizar las métricas BLEU y ROUGE para evaluar el agente RAG (se utilizarán para tener un punto de partida para comparar los resultados del agente, ya que se considera que la capacidad para generar texto nuevo y creativamente de los LLMs usados está ya testada y es buena).

En esta primera iteración se ha obtenido una puntuación media de BLUE de 0.0854, este resultado es excesivamente bajo (debido a que se ha introducido una única frase en el conjunto de respuestas esperadas).

Por otro lado, la puntuación media obtenida para ROUGE es de 0.2539 para ROUGE-1, indicando que existe una superposición de unigramas del 25.39%. Una puntuación media de 0.1447 en ROUGE-2, lo que indica una superposición de bigramas del 14.47% y una puntuación media de 0.2539 en ROUGE-l por lo que el texto generado tiene una similitud del 25.39% con el texto de referencia. En definitiva, los resultados obtenidos en estas métricas son moderados y bajos.

Por último, el tiempo medio de respuesta es de 1176.9203 segundos excesivamente alto, esto es debido a la herramienta de resumen que como se explicó en el marco teórico, esta herramienta accede a todos los nodos lo que aumenta el tiempo medio de respuesta.

2.2.5 Segunda Iteración

2.2.5.1 Preproceso

En esta segunda iteración, se pretende generar un agente RAG multi-documento, introduciendo un nuevo conjunto de datos en la base de conocimiento del agente. Además, se sustituirá el LLM utilizado en la sección anterior por una conexión a la API groq pudiendo seleccionar diferentes modelos dentro del Agente RAG, lo que aportará una mayor flexibilidad. Por último, se pretende implementar la elaboración de la aplicación web.

En esta segunda iteración, en un principio, se tomará como punto de partida la base de conocimiento del sprint anterior y se expandirá añadiéndole un documento construido a partir del *dataset* relativo a los prompts text-to-video.

Se ha decidido no realizar ningún tipo de limpieza de datos dentro del *dataset*, ya que se considera que es una fuente completa y sin errores significativos para completar la base de conocimiento.

Por tanto, en el script *text2video_prompts.py* se realiza una serie de transformaciones para extraer únicamente las descripciones reales de prompts text-to-video. Además, una vez unificado se ha añadido una primera línea en la que se describe el contenido del documento, siguiendo la lógica utilizada en la primera iteración.

Finalmente, se convierte la serie completa en un archivo de texto que será el que forme la base de conocimiento del agente.

El preproceso de esta segunda iteración no termina en este punto ya que debido a que este agente RAG multi-documento genera tiempos de ejecución muy elevados, se ha decidido generar un agente monodocumento con una base de conocimiento de un documento que unifica los documentos de la base de conocimiento anterior.

Por lo cual, el script *merged.py* crea y unifica en un único documento *merged.txt* los dos documentos creados con anterioridad. Cabe destacar que el archivo *text2video_prompts.py* sufre una modificación particular que ayuda a reducir el tiempo de ejecución posterior del agente. En el programa, una vez extraídos los prompts, se reduce la serie a 1000 ejemplos por lo que el documento tendrá un tamaño similar al número de ejemplos de anuncios, lo que hace que los dos factores de condicionamiento estén equilibrados. Se ha elegido escoger las 1000 primeras apariciones ya que el orden dentro del *dataset* inicial es totalmente aleatorio.

2.2.5.2 Modelado

En la primera iteración se decidió fijar en un único modelo (Zephyr-7b) el LLM en el que se apoya el agente RAG, siendo está una implementación rígida para el usuario. Además, la descarga de este LLM requiere del uso de GPUs por lo tanto requiere de una infraestructura de procesamiento avanzada.

Para solucionar estas limitaciones y dotar de mayor flexibilidad en la configuración del agente, en esta segunda iteración se ha decidido utilizar la API de groq para implementar los distintos LLMs que ofrece, estableciendo el modelo *mixtral-8x7b* como predeterminado, ya que es el que ofrece mejores características para la consecución del objetivo.

Groq ofrece una API gratuita que para su utilización únicamente requiere de un API_KEY, el cual se consigue creando una cuenta en dicha plataforma de manera gratuita también. Además, debido a la manera en la que el LLM de la primera iteración estaba implementado, el agente era del tipo ReAct (reason + act) el cual integraba una capa de razonamiento y ejecución de acciones. Con la API de groq se ha decidido utilizar un agent worker y un agent runner proporcionando una implementación más clara (de más bajo nivel) y acorde con el marco teórico.

Por otro lado, en esta segunda iteración se decide eliminar la herramienta de resumen del agente debido a que el principal objetivo del agente RAG que se construye es crear contenido nuevo y creativo (generación/expansión de prompts text-to-video orientados a anuncios) por lo que generar un resumen de alguno de los documentos de la base de conocimiento no tiene sentido (ya que son ejemplos de anuncios). Con esta eliminación se reducirá el tiempo de construcción del agente RAG, lo que es muy necesario como hemos podido ver en la iteración anterior.

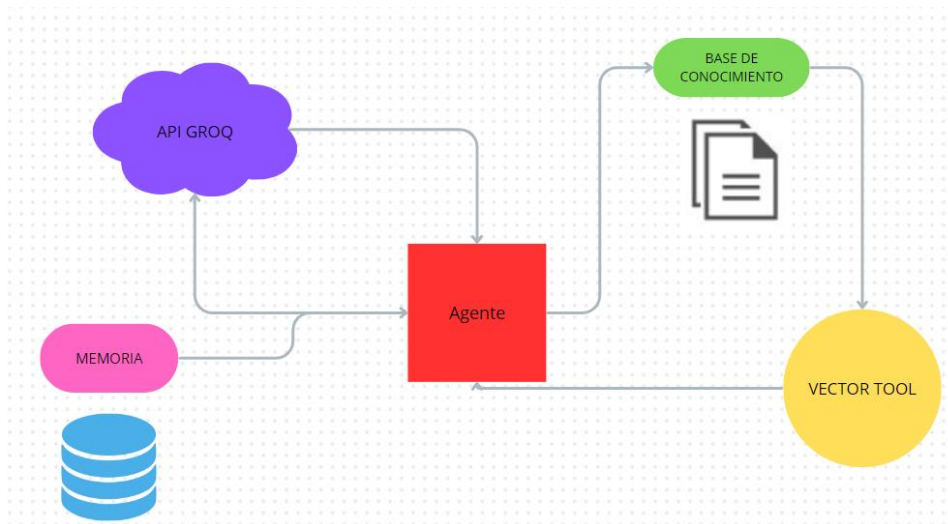


Figura 10: Implementación del agente final (Fuente: Creación propia)

A su vez, para una mejora en la precisión y refinamiento del agente se ha introducido un “system prompt” (prompt del sistema) especificando la función del agente, su objetivo principal y el formato de salida de las respuestas. Para ello se han utilizado palabras claves que se han ido consiguiendo y configurando mediante un proceso iterativo de prueba y error.

Posteriormente, durante la fase de evaluación y con el afán de intentar mejorar aún más la precisión del agente se implementó un conjunto de ejemplos de pares query-respuesta para realizar few-shot learning (habilidad del modelo para generalizar después de haber sido entrenado con un grupo pequeño de ejemplos).

Evalutando esta propuesta finalmente se decidió descartarla ya que no mejoraba los resultados anteriores por tanto con la implementación final, el agente RAG no requiere de un conjunto de ejemplos para realizar few-shot learning, sino que lo realiza mediante zero-shot learning dependiendo únicamente en el transfer learning.

2.2.5.3 Evaluación

En esta segunda iteración únicamente se va a evaluar la implementación del agente, ya que la elección del LLM recae sobre el usuario.

Las métricas usadas en esta segunda iteración son las mismas que las usadas en la primera, pero se debe de tener en cuenta que las herramientas de resumen se han eliminado por tanto esa capacidad respecto a la iteración anterior no se va a evaluar. Se ha obtenido un average BLEU Score de 0.1125 mejorando a la primera iteración, algo esperado ya que la base de conocimiento es más amplia. Esta métrica sigue siendo muy baja debido a la misma razón que en la primera iteración, la falta de introducción de prompts de referencia a la hora de evaluar el agente RAG, pero como usaremos estas métricas como referencias para comparar los distintos agentes RAG y se presupone una buena generación de texto por los LLMs usados no introduciremos más textos de referencia (ya que podríamos falsear la evaluación).

Por otro lado, en las métricas rouge, se ha obtenido una puntuación media 0.2909 en ROUGE-1 indicando que existe una superposición del 29.09% de unigramas entre el texto generado y el texto de referencia, lo cual es moderado, pero no muy alto. En ROUGE-2 se ha obtenido una puntuación media de 0.1967 indicando que existe una superposición del 19.67% de bigramas, lo que es muy bajo, lo que sugiere que existe dificultades para captar relaciones de palabras a corto plazo en comparación con el texto de referencia. En cuanto a la puntuación media para ROUGE-L es de 0.2909 que sugiere que el texto generado tiene una similitud del 29.09% con el texto de referencia.

Por último, se ha obtenido un tiempo promedio de respuesta de 12.4989 segundos, lo que es un tiempo excesivamente alto. Esto se debe principalmente a que el agente hace uso de una base de conocimiento con múltiples documentos y en especial el documento relativo a prompts text-to-video es un documento grande por lo que si hace uso de esa herramienta el tiempo de respuesta puede ser demasiado elevado.

Por ello y, por la necesidad de utilizar tanto el documento relativo a anuncios y el documento relativo a prompts text-to-video para generar cada respuesta, se ha decidido unificar los documentos y construir una base de conocimiento de un solo documento y, se ha vuelto a evaluar el agente.

En esta nueva evaluación se ha obtenido un average BLEU score de 0.2477, la cual mejora a la anterior, algo esperado al unificar los documentos.

Para las métricas rouge, los resultados son los siguientes:

- Average ROUGE-1: 0.4848
- Average ROUGE-2: 0.3529
- Average ROUGE-L: 0.4848

Las cuales mejoran también a las anteriores.

Por último, con esta modificación se reduce extensiblemente el tiempo medio de ejecución a 9.8171 segundos.

2.2.5.4 Elaboración de la interfaz web

A continuación, se van a detallar las fases seguidas para la elaboración de la aplicación web. Esta aplicación se ha llamado “Video Ads Generator”.

Análisis de requisitos

Perspectiva del producto: “Video Ads Generator” es una aplicación diseñada para ayudar a los usuarios menos especializados en IA a generar anuncios de video de manera semiautomática y después visualizarlos. El objetivo principal es proporcionar una interfaz intuitiva que facilite la creación de las descripciones necesarias para generar un anuncio de video con IA proporcionando a su vez un interfaz para visualizar los resultados. Con el tiempo y, mayor inversión económica se esperaría una evolución a poder integrar modelos generativos de video en la propia aplicación para poder así generar los anuncios de manera automática.

Restricciones:

- Tecnológicas: La aplicación debe ser compatible con los navegadores de mayor uso como Google Chrome, Mozilla Firefox o Safari. Además, debe estar optimizada para dispositivos móviles y ordenadores.
- Rendimiento: La generación de las descripciones se debe de realizar de manera eficiente para no afectar la experiencia del usuario. El objetivo es que los tiempos de carga (del chatbot sobre todo) estén entre 10 y 20 minutos.

Además, el sistema se deberá desarrollar en el lenguaje Python y se deberá tener una buena conexión a internet para usar la aplicación.

Requisitos de interfaz:

- RI1: El sistema debe proporcionar una página inicial (página de control) para acceder a las distintas funcionalidades de la aplicación.
- RI2: El sistema debe proporcionar una página interactiva estilo “ChatBot”, para que el usuario introduzca las características del anuncio a generar, almacenando la conversación.
- RI3: El sistema debe de proporcionar una página para cargar (desde local) el video generado (web pika.ai u otra a gusto del usuario).

Requisitos funcionales:

- RF1: El sistema debe permitir cargar un video en diferentes formatos: mp4, MOV, MPEG4, etc.
- RF2: El sistema debe permitir mostrar el video descargado.
- RF3: El sistema debe permitir generar una extensión del texto introducido como input en el chat.
- RF4: El sistema debe indicar el modelo elegido para integrarse en el agente.
- RF5: El sistema debe mostrar desplegables para elegir modelo e insertar videos

Requisitos no funcionales:

- RNF1: El sistema debe adaptarse automáticamente a diferentes tamaños de pantalla y dispositivos (diseño *responsive*).
- RNF2: El tiempo de carga debe ser inferior a 30 minutos y el tiempo de respuesta debe de ser inferior a 5 minutos.
- RNF3: El sistema debe ser completamente funcional en todo tipo de dispositivos tanto ordenadores tradicionales como en dispositivos táctiles.
- RNF4: El sistema debe ser accesible en inglés principalmente.

Seguridad: El sistema no requiere de autenticación de usuarios ni de autorización de acceso a funcionalidades específicas. Los datos utilizados por el sistema deben de ser almacenados y transmitidos de forma segura.

Diseño de la aplicación

Descripción del propósito del sistema: El sistema dará acceso al servicio de generación de prompts text-to-video orientados a la generación de anuncios de video. Además, proporcionará un espacio para visualizar los videos generados con esos prompts utilizando alguna herramienta externa gratuita disponible (se recomienda pika.ai). Se busca simplicidad en el proceso de generación de anuncios de video, eliminando la necesidad de conocimientos técnicos especializados en IA. Los usuarios pueden escribir en lenguaje natural y, la aplicación se encarga de devolver la descripción necesaria para elaborar el video con un modelo text-to-video.

**Nota:* Se ha intentado integrar la herramienta de generación de video, pero debido a la novedad de estos sistemas, todas las APIs son de pago.

Lista de acontecimientos: A continuación, se van a enumerar los distintos eventos clave que pueden ocurrir durante la interacción del usuario con la aplicación.

- Acontecimiento 1: El usuario entra en la página del chatbot.

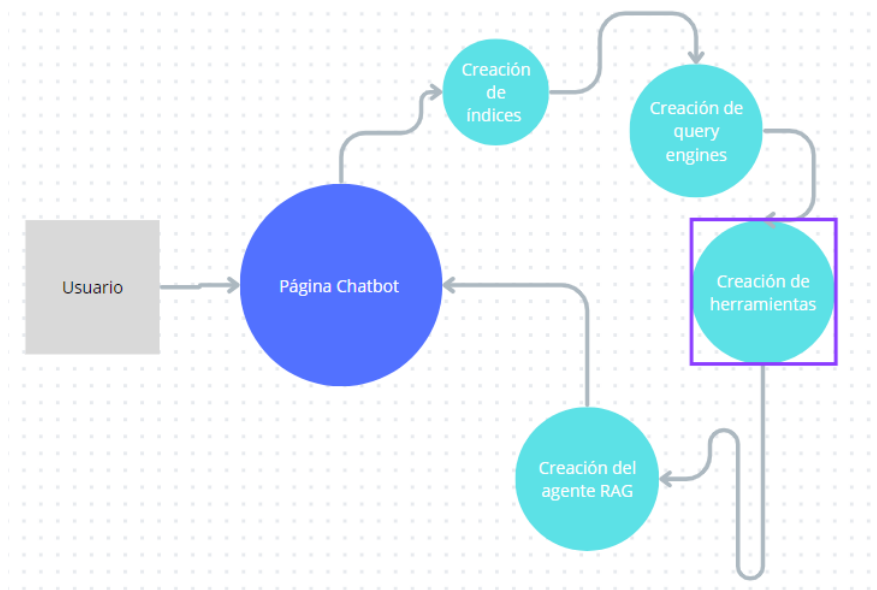


Figura 11: Acontecimiento 1 detallado (fuente: Creación propia)

- Acontecimiento 2: El usuario introduce un prompt en el chat y por tanto se genera una respuesta.

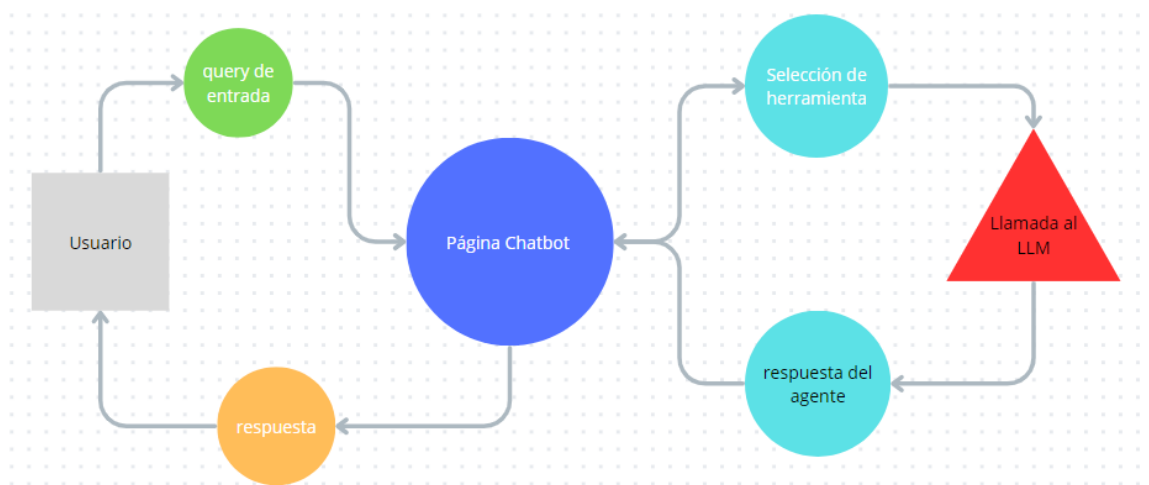


Figura 12: Acontecimiento 2 detallado (fuente: Creación propia)

- Acontecimiento 3: El usuario entra en la página de video.

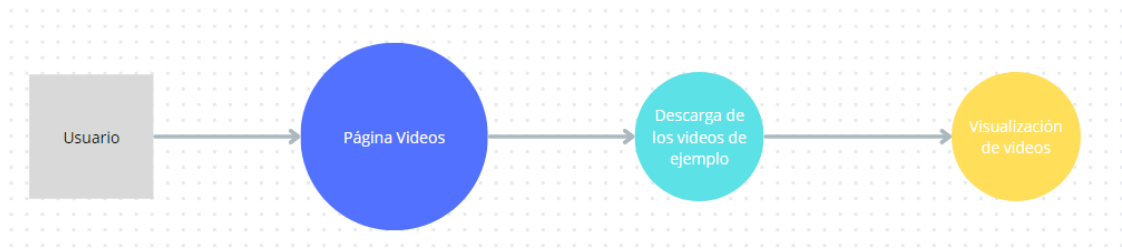


Figura 13: Acontecimiento 3 detallado (fuente: Creación propia)

- Acontecimiento 4: El usuario carga un video en la página.

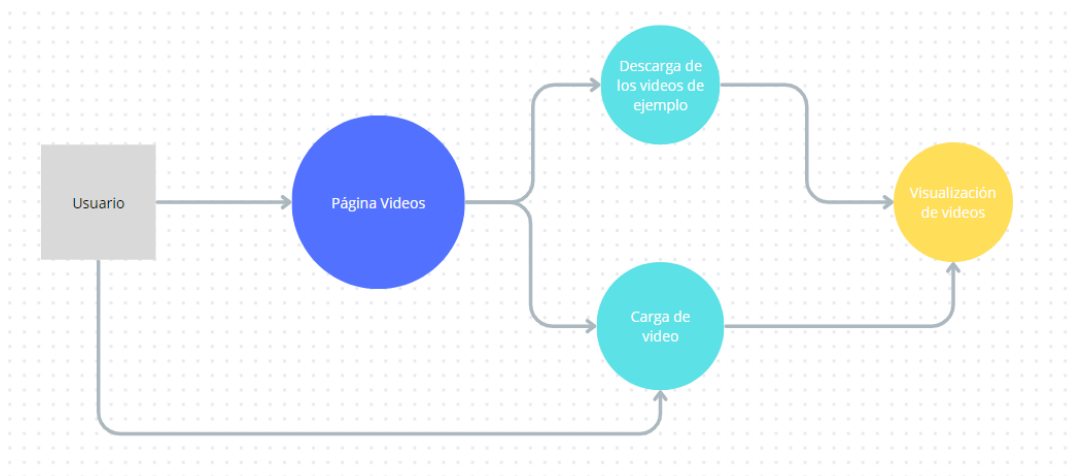


Figura 14: Acontecimiento 4 detallado (fuente: Creación propia)

Matriz de trazabilidad: La matriz de trazabilidad mapea los requisitos funcionales con las funcionalidades implementadas en la aplicación, asegurando que los requisitos definidos en la fase de análisis sean abordados adecuadamente durante el diseño y la implementación.

	Acont. 1	Acont. 2	Acont. 3	Acont. 4
RF1			x	x
RF2			x	x
RF3	x	x		
RF4	x			
RF5	x			x

Figura 15: Matriz de trazabilidad (Fuente: Creación Propia)

Diseño de interfaz: La interfaz propuesta para satisfacer los requisitos y desencadenar los acontecimientos es la siguiente.

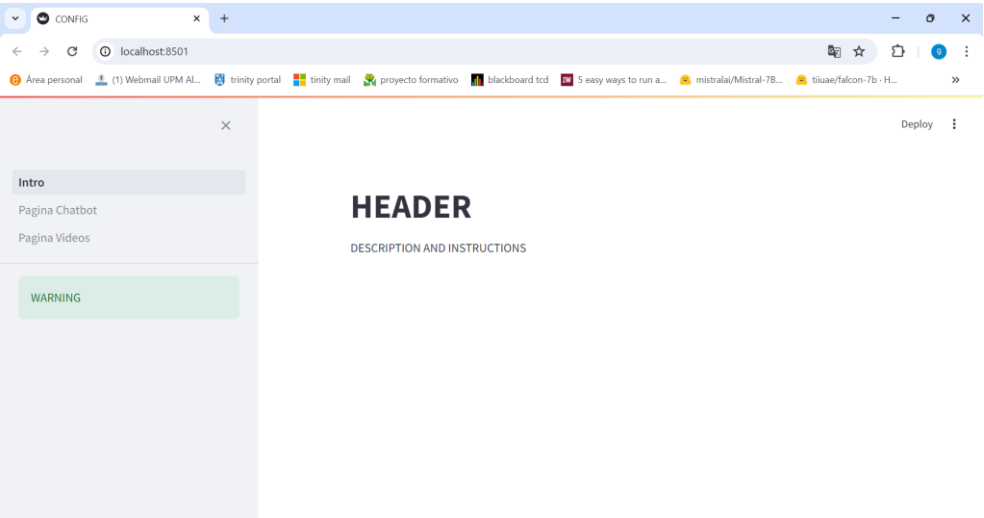


Figura 16: Diseño de la página de inicio (fuente: Creación propia)

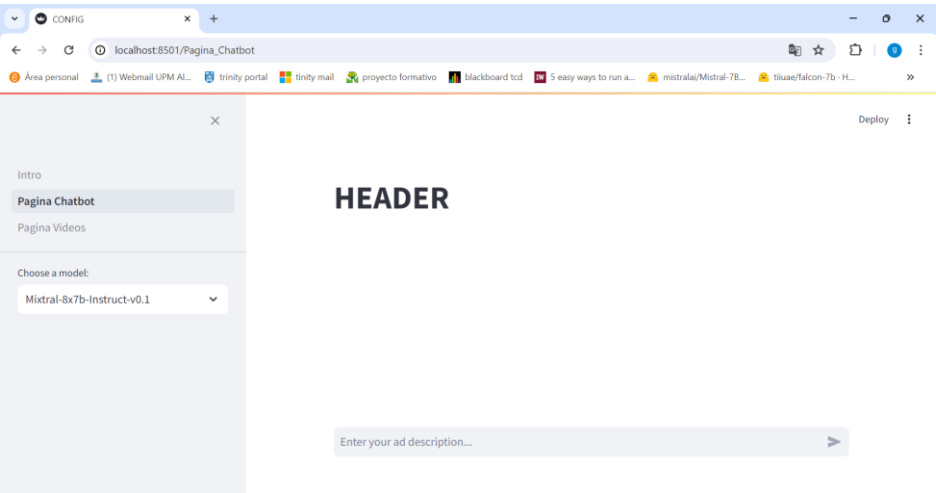


Figura 17: Diseño página ChatBot (fuente: Creación propia)

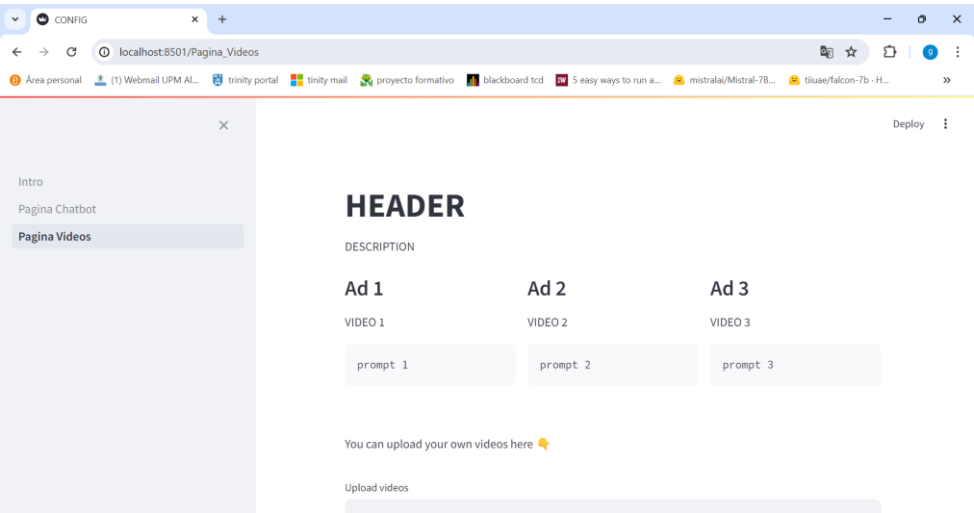


Figura 18: Diseño de la página de visualización de videos (fuente: Creación propia)

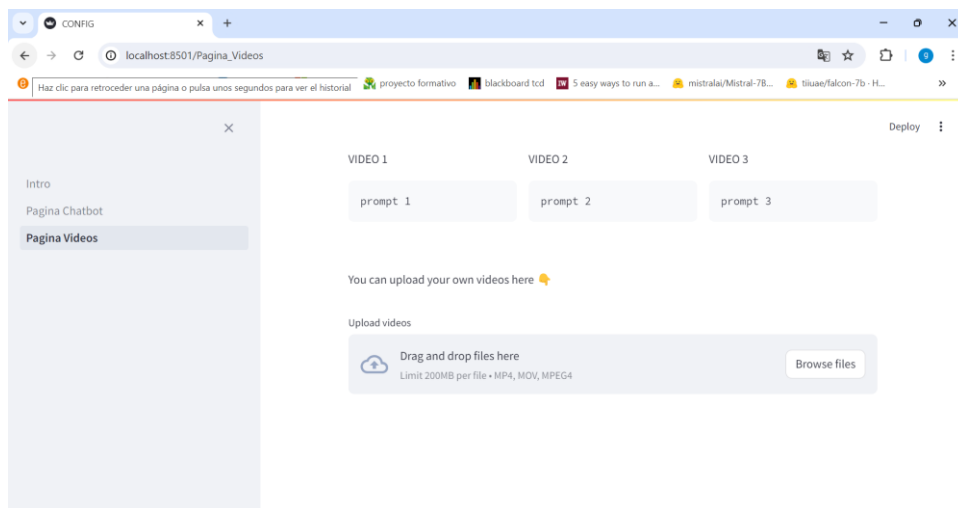


Figura 19: Diseño de la página de visualización de videos 2 (fuente: Creación propia)

Codificación

La implementación de la aplicación web se desarrolla completamente en Python, utilizando la librería *streamlit* y las funciones predefinidas dentro de ella para construir la interfaz. Además, se utilizará la versión *streamlit cloud* para subir la aplicación a la nube. Para esta función se requiere de un repositorio de GitHub el cual se enlazará con *streamlit cloud* para poner la aplicación a disposición en la nube.

Se ha utilizado como programa principal el archivo *Intro.py* el cual da la bienvenida al usuario y describe la aplicación y su uso brevemente. Por otro lado, al ser una aplicación multi página, se han guardado los archivos *chatbot.py* y *video.py* en una carpeta *pages*. De esta forma *streamlit* entiende que se está creando una aplicación multi página e implementa las funciones necesarias. Por último, tanto la página del chatbot como la página de visualización de videos hacen uso de las funciones auxiliares implementadas en la carpeta *utils.py*. Además, se deben incluir una carpeta *.streamlit* con un archivo *secrets.toml* que contiene la clave de acceso a la API de groq, un archivo con las bibliotecas a instalar *requirements.txt* y otro archivo *merged.txt* que compone la base de conocimiento del agente RAG, así como una carpeta *videos* con los videos generados de ejemplo.

En conclusión, se ha implementado la interfaz tanto localmente como en la nube siendo *streamlit* el actor principal para llevar a cabo las dos tareas. El repositorio de GitHub utilizado es un repositorio privado por lo que la aplicación será implementada en ámbito privado y deberá hacerse pública manualmente. Para ello, *streamlit cloud* proporciona una interfaz donde se pueden realizar estos cambios y monitorizar la aplicación en la nube.

Pruebas

A continuación, se listan las pruebas de integración realizadas:

- Pruebas unitarias: Se ha comprobado que los desplegables y los botones responden a las acciones clic y, que los campos de texto del chatbot y el widget para subir videos funcionan.
- Pruebas de dispositivos y navegadores: Se ha probado que la funcionalidad de la aplicación es correcta en los navegadores Google Chrome, Microsoft Edge, Safari y Mozilla Firefox. Además, se ha comprobado que el funcionamiento es correcto tanto en dispositivos táctiles como ordenadores.
- Pruebas de rendimiento: Se ha medido el tiempo de carga en el chatbot, el cual es bastante elevado, pero está dentro del intervalo requerido. Además, se ha comprobado que los tiempos de carga de video son rápidos. Por otro lado, los tiempos de respuesta una vez cargado el chatbot son aceptables y rápidos.
- Pruebas de integración: Se ha simulado un uso típico de la aplicación desde la carga del chatbot, introducción de un prompt corto, edición de respuesta y, por otro lado, carga de video en la aplicación asegurando que cada etapa se completa sin errores.

Producto final

A continuación, se muestra el producto final:

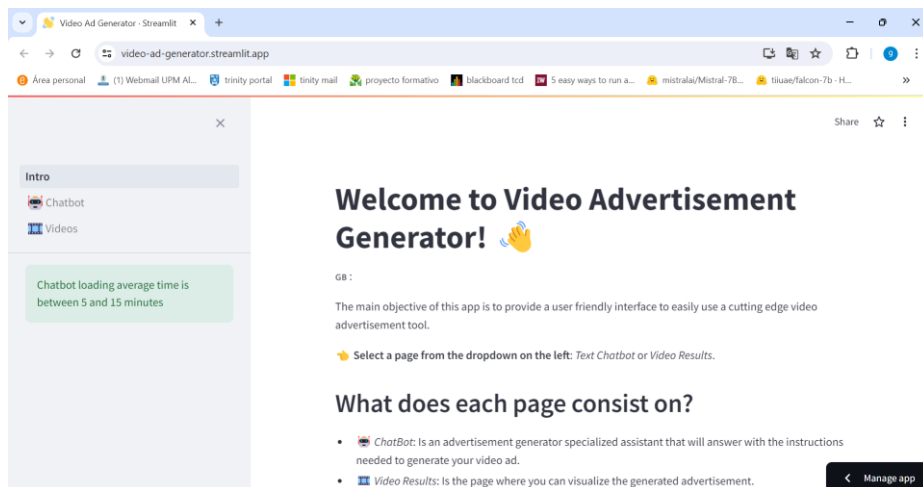


Figura 20: Página de Inicio (fuente: creación propia)

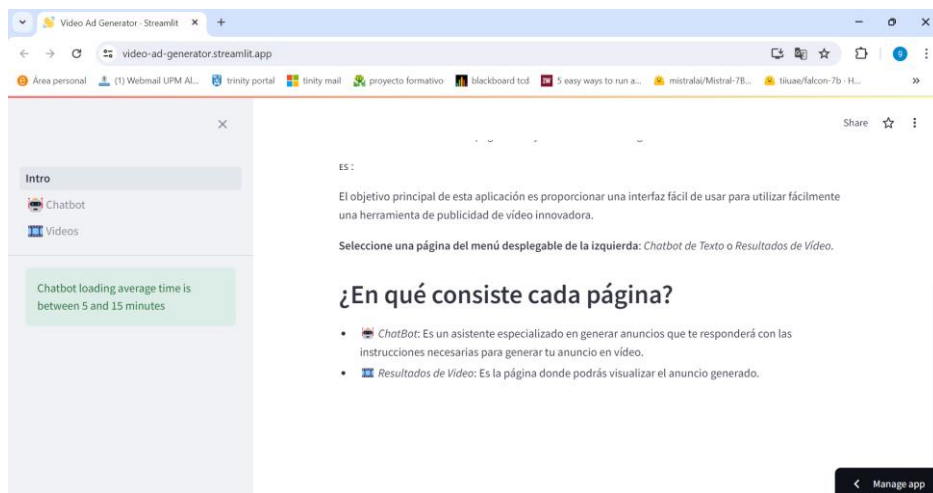


Figura 21: Página de Inicio 2 (fuente: creación propia)

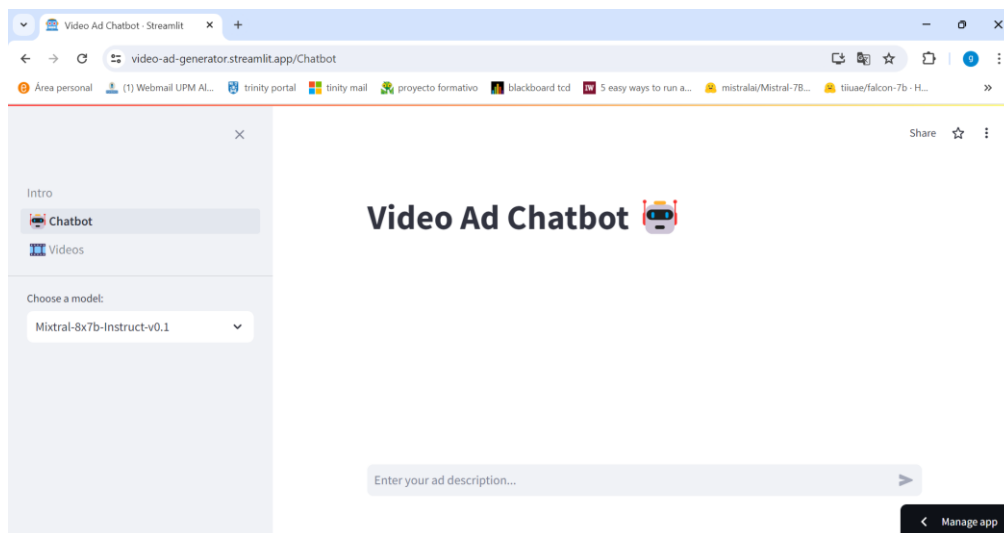


Figura 22: Página del ChatBot (fuente: Creación propia)

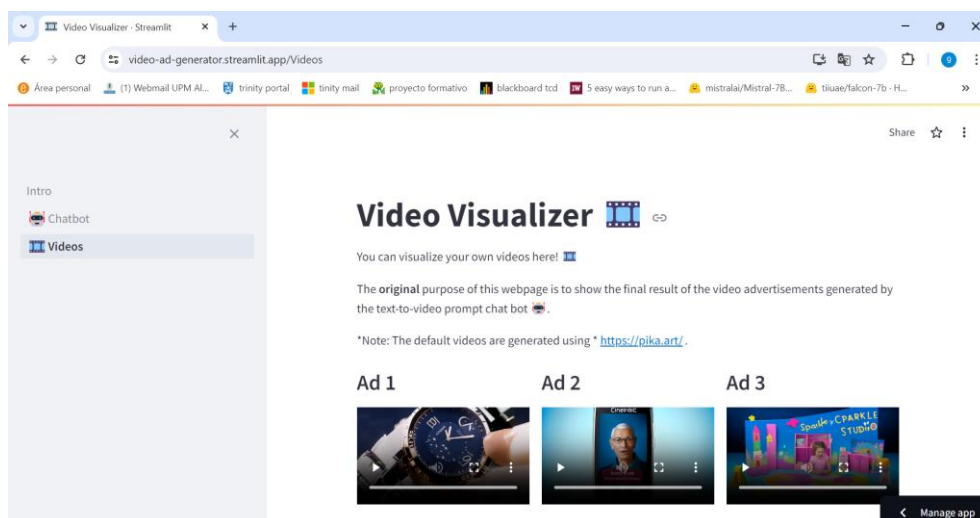


Figura 23: Página Visualizador de Videos (fuente: Creación propia)

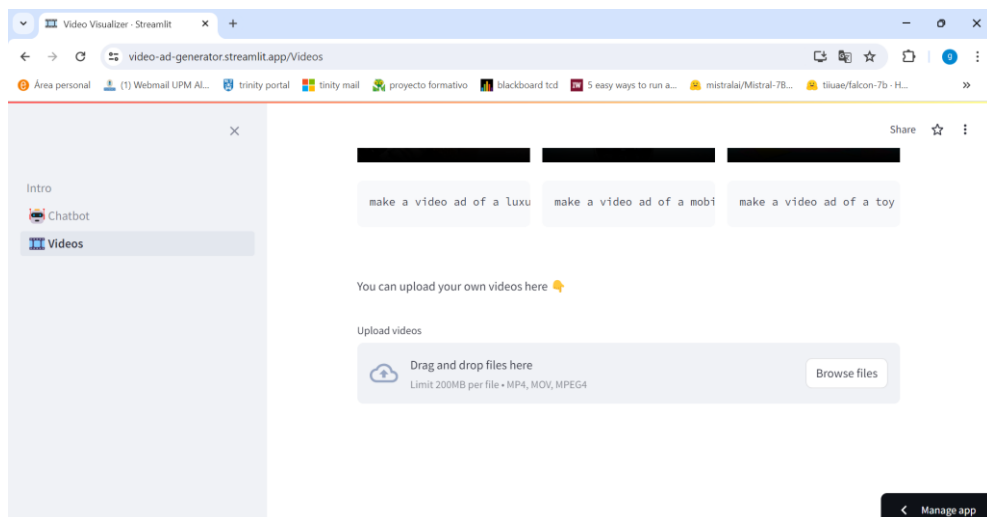


Figura 24: Página Visualizador de Videos 2 (fuente: Creación propia)

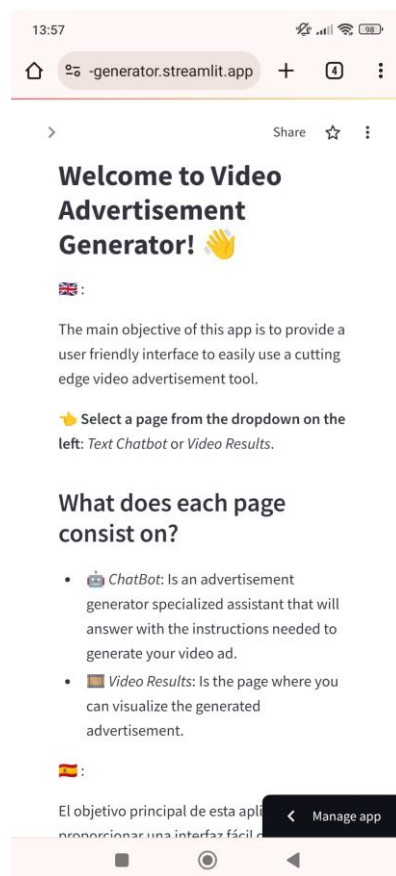


Figura 25: Página de inicio Móvil (fuente: Creación propia)

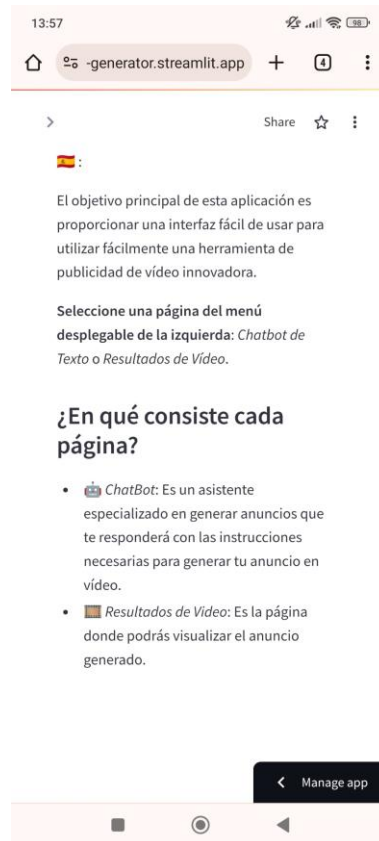


Figura 26: Página de inicio Móvil 2 (fuente: Creación propia)

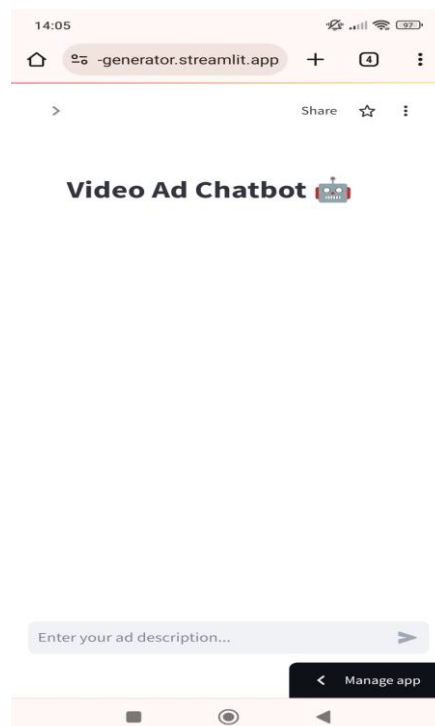


Figura 27: Página de ChatBot móvil (fuente: Creación propia)

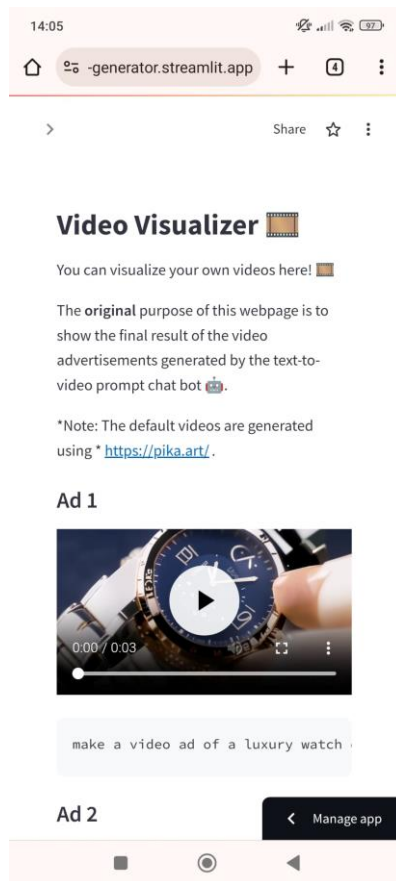


Figura 28: Página de video móvil (fuente: Creación propia)

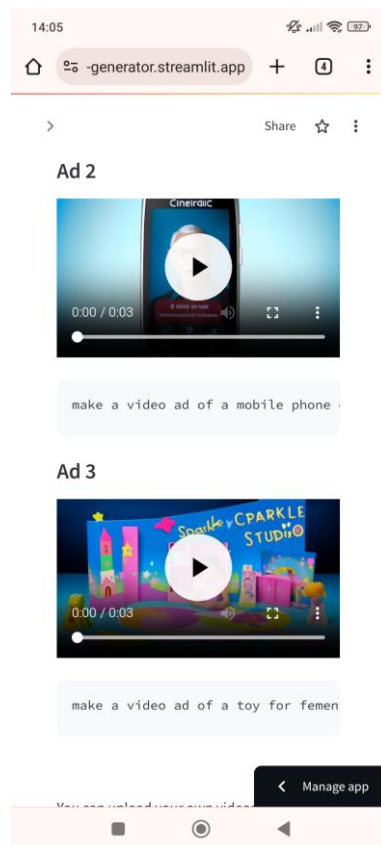


Figura 29: Página de video móvil 2 (fuente: Creación propia)

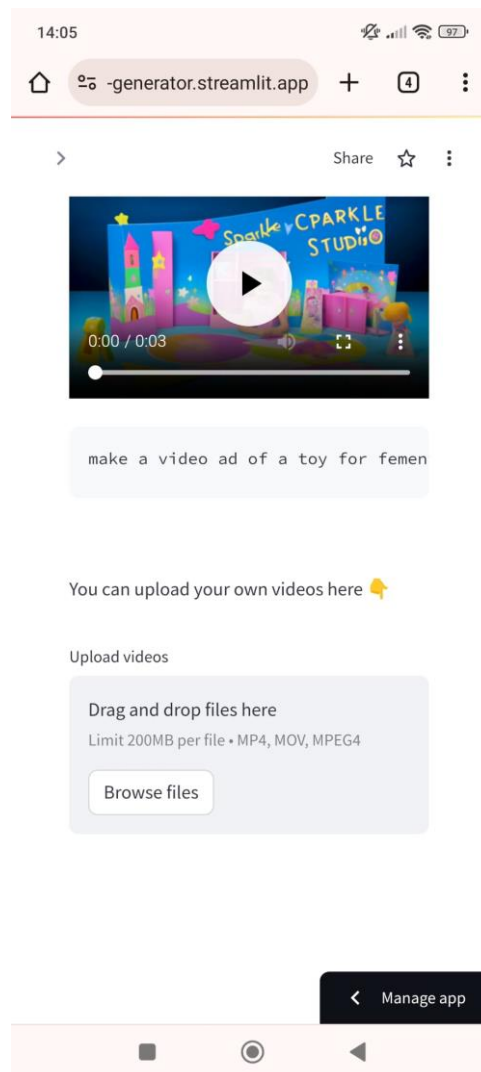


Figura 30: Página de video móvil 3 (fuente: Creación propia)

3 Resultados y Conclusiones

3.1 Discusión

3.1.1 Comparación con trabajos relacionados e interpretación de los resultados obtenidos

A lo largo del trabajo realizado y como resultado del proceso seguido se ha logrado implementar un sistema de ayuda a usuario especializado en la generación de prompts text-to-video orientados a crear anuncios de video personalizados, además, de una aplicación web que permite a personas no expertas en materia de IA generar descripciones detalladas para crear anuncios de video con modelos de IA generativa y visualizar los resultados de manera totalmente gratuita y accesible. Aunque necesite de un poco de tiempo para entender su funcionamiento y sea un poco ardua la generación de los videos se propone una herramienta al alcance de todo el mundo que genera resultados detallados y precisos para la generación de anuncios de video.

Existen diferentes aproximaciones a la herramienta propuesta que ofrecen resultados de mayor calidad y de mayor simplicidad a la hora de generar el video directamente, pero estas o incluyen métodos de pago, lo que las hace menos accesibles o requieren de una gran capacidad de recursos computacionales para ejecutarlas por lo que también las hace menos accesibles.

A pesar de que el rendimiento, en cuanto a tiempo de carga es muy lento, los resultados finales son prometedores ya que se ha conseguido que el asistente virtual genere instrucciones precisas para crear anuncios de video cortos, utilizando la base de conocimiento proporcionada. Por supuesto, los resultados serían mejores si se hubieran utilizado recursos computacionales avanzados y se pudiera haber hecho fine-tuning sobre un conjunto de datos compuesto por pares {query: respuesta} pero esta solución alternativa y creativa no se queda muy lejos de los resultados que se obtendrían de esta otra forma.

Cabe destacar que, aunque los resultados obtenidos en métricas BLUE y rouge son muy bajos y ampliamente mejorables la realidad es que el agente RAG cumple con el objetivo para el que se ha construido devolviendo respuestas coherentes y precisas a preguntas del estilo “quiero generar un anuncio de video ...”. Estas respuestas siguen el formato de lista de instrucciones lo que hacen sencilla la lectura y comprensión de la respuesta.

Además, el tiempo de ejecución es bajo por lo que la velocidad de respuesta es un elemento que destacar del agente RAG final.

Por otro lado, se recomienda usar los siguientes prompts para maximizar el rendimiento del agente:

- “I want to create a video ad of ...”
- “Act like an expert in ... and build a video ad ...”
- “Build a video ad with the following features ...”

3.1.2 Limitaciones y posibles mejoras

Al ser una aplicación desarrollada para que pueda ser utilizada gratuitamente y, estar implementándose una tecnología muy novedosa como se explicaba al comienzo de este trabajo, el acceso a ciertas funcionalidades es muy limitado ya que es de pago. Esto obviamente hace que la aplicación tenga algunas limitaciones, las cuales se han ido comentando a lo largo del trabajo. Las limitaciones principales son:

En primer lugar, la creación del anuncio no es automática, es semi automática ya que se necesita utilizar una herramienta externa para la creación del video por lo que se necesita intervención humana para ello. Para mejorarlo se propone utilizar alguna API de pago de estos modelos o si se tienen recursos computacionales suficientes implementar un modelo de video directamente en el pipeline y así automatizar por completo la creación del anuncio de video. Otra solución un poco más creativa sería escribir un código que mediante web scrapping accediera a las webs que proporcionan estas herramientas de creación de video y creara el video y lo descargara.

En segundo lugar, el tiempo de carga del asistente virtual es muy elevado y debería reducirse para mejorar la experiencia del usuario. Para ello se propone como solución utilizar un “vector store” o base de datos vectorial en la nube para guardar los índices creados por el agente RAG. Así no tendrían que crearse todas las veces que se quisiera usar, reduciendo el tiempo de carga del asistente.

Por último, la última gran limitación que se aprecia es la necesidad de depender de un agente como streamlit cloud para el mantenimiento del servidor en la nube, el cual se sobrecarga y falla cuando hacen uso de la aplicación unos pocos usuarios, ofreciendo como solución el hacer un reboot del servidor. Como solución se propone contratar un proveedor privado estilo AWS u Oracle para poder controlar la tolerancia del servidor.

3.1.3 Vinculación con los objetivos de desarrollo sostenible

Analizando el trabajo en relación con los 17 Objetivos de Desarrollo Sostenible propuestos por la ONU e incluidos en la Agenda 2030, se puede observar las siguientes repercusiones.

Para empezar, es importante saber que, como bien su nombre indica, estos objetivos abalan por la igualdad entre individuos, así como la protección del planeta y la certeza de constituir una sociedad global próspera y pacífica en el horizonte 2030.

Se encuentra una clara relación entre el trabajo realizado con algunos de los objetivos en específico. Por ejemplo, refiriéndose al ODS número 4, que habla sobre una educación de calidad. La herramienta diseñada no sólo puede incluirse como una ayuda a la hora del trabajo empresarial, sino también como parte de una guía de estudios de calidad, que mantenga tanto a estudiantes como a propios trabajadores en formación, en continuo proceso de aprendizaje sobre las innovaciones tecnológicas que se desarrollan diariamente.

Continuando con el ODS número 8 que se refiere al trabajo decente y el crecimiento económico. Una de las principales causas del crecimiento económico se trata de la productividad del trabajo.

El sistema diseñado no sólo influye en la calidad de las habilidades de los trabajadores, sino que también incrementa la productividad de los mismos al tener un beneficio en el tiempo invertido, reduciéndolo y permitiendo un avance más rápido.

En el ODS número 9, que se refiere a la industria, innovación e infraestructuras, se puede observar la relación fácilmente. La aplicación desarrollada se trata sobre todo de una innovación tecnológica, una mejora del proceso de producción de anuncios.

Y por último se encuentra la ODS número 10, la cual lucha por la reducción de desigualdades. La herramienta diseñada se trata de una aplicación de carácter público, es decir, el uso de esta no viene determinado por el poder adquisitivo de la población. Esto crea un escenario en el que cualquier persona desde cualquier dispositivo electrónico puede acceder a ella sin discernir entre niveles económicos.

El resto de los objetivos, la salud y bienestar, la paz, justicia e instituciones sólidas, la producción y consumo responsables, etc., no se ven afectados de una forma directa por la existencia de este nuevo sistema, sin embargo, el uso de este sí puede contribuir a alcanzarlas. Por ejemplo, si una empresa de carácter no lucrativo que busca sensibilizar a la sociedad sobre la igualdad de género y decide mediante esta aplicación desarrollar una campaña de concienciación sobre la equidad, estaría apoyando el ODS 5 que nos habla de la igualdad de género, de una forma indirecta.

3.2 Conclusiones y Futuras Líneas de Investigación

En este trabajo se ha implementado un sistema que facilita la generación de prompts text-to-video para la creación de anuncios mediante el uso de IA generativa, diseñándose una aplicación web intuitiva que acerca a los usuarios no especializados en IA a crear descripciones detalladas para anuncios de video, visualizando los resultados de manera gratuita. Aunque el sistema necesita mejoras en temas de mantenimiento de servidor, tiempos de carga y automatización completa del proceso los resultados muestran el potencial de la herramienta ofreciendo su uso a cualquier usuario con acceso a internet.

El proyecto ha logrado todos los objetivos principales del trabajo donde, se ha realizado un proceso de investigación y documentación de tecnologías relacionadas con la inteligencia artificial generativa, entendiendo en profundidad las diferentes arquitecturas, su funcionamiento y sus características. Entendiendo como se implementaban en trabajos relacionados y como podían influir en este proyecto y, diseñando una metodología de trabajo adecuada para poder ejecutar y desarrollar un proceso de desarrollo de software e investigación de forma conjunta. Sin embargo, se han identificado unas limitaciones, como la necesidad de intervención humana, lo que da pie a futuras mejoras.

Para futuras investigaciones, se recomienda explorar como se añadir una personalización individual de los anuncios generados al modelo expandiendo así sus funcionalidades. Esto incluiría datos de clientes, su comportamiento de navegación e interacciones en redes sociales. Por otro lado, se propone añadir al chatbot una optimización multimodal para que pueda integrar otro tipo de datos como el audio o la interacción táctil para que pueda generar anuncios más envolventes mejorando así la experiencia del usuario.

Estas dos líneas propuestas no solo ampliarían el conocimiento actual sobre la generación de anuncios mediante IA generativa, sino que también ofrecerán nuevas herramientas y enfoques para mejorar la efectividad en el mundo de la publicidad.

4 Bibliografia

- [1] H. Woo Park, C. Velasco Lim, M. Omar, and Y. Peng Zhu, 'Decoding the Relationship of Artificial Intelligence, Advertising, and Generative Models', Jan. 2024, [Online]. Available: <https://www.preprints.org/manuscript/202401.0373/v1>
- [2] J. D. Brüns and M. Meißner, 'Do you create your content yourself? Using generative artificial intelligence for social media content creation diminishes perceived brand authenticity', *J. Retail. Consum. Serv.*, vol. 79, p. 103790, 2024, doi: <https://doi.org/10.1016/j.jretconser.2024.103790>.
- [3] N. Kshetri, Y. K. Dwivedi, T. H. Davenport, and N. Panteli, 'Generative artificial intelligence in marketing: Applications, opportunities, challenges, and research agenda', *Int. J. Inf. Manag.*, vol. 75, p. 102716, 2024, doi: <https://doi.org/10.1016/j.ijinfomgt.2023.102716>.
- [4] E. Gołab-Andrzejak, 'The Impact of Generative AI and ChatGPT on Creating Digital Advertising Campaigns', *Cybern. Syst.*, pp. 1–15, doi: 10.1080/01969722.2023.2296253.
- [5] M. Kumar and A. Kapoor, 'Generative AI and Personalized Video Advertisements', *SSRN*, Nov. 2023, [Online]. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4614118
- [6] J. Winiarski, J. Januszewicz, and P. Smoliński, 'Towards completely automated advertisement personalization: an integration of generative AI and information systems', *Information Systems Development, Organizational Aspects and Societal Trends*, p. 9, 2023, doi: 10.62036/ISD.2023.60.
- [7] D. L. Hocutt, 'Composing with generative AI on digital advertising platforms', *Comput. Compos.*, vol. 71, p. 102829, 2024, doi: <https://doi.org/10.1016/j.compcom.2024.102829>.
- [8] B. Adelakun, 'The Fusion of Creativity and Technology: Generative Artificial Intelligence Tools for Marketing', 2023, [Online]. Available: <https://www.theseus.fi/handle/10024/815346>
- [9] S. Bhausheb Zinjad, A. Bhattacharjee, A. Bhilegaonkar, and H. Liu, 'ResumeFlow: An LLM-facilitated Pipeline for Personalized Resume Generation and Refinement', Feb. 2024, [Online]. Available: <https://paperswithcode.com/paper/resumeflow-an-llm-facilitated-pipeline-for>
- [10] R. Ma *et al.*, 'Poster: PipeLLM: Pipeline LLM Inference on Heterogeneous Devices with Sequence Slicing', in *Proceedings of the ACM SIGCOMM 2023 Conference*, in ACM SIGCOMM '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 1126–1128. doi: 10.1145/3603269.3610856.
- [11] Dr. A. Majumdar *et al.*, 'Building Pipelines and Environments for Large Language Models', *Mphasis*, [Online]. Available: <chrome-extension://efaidnbmnnnibpcajpgclefindmkaj/https://www.mphasis-ai.com/content/dam/mphasis-com/global/en/home/innovation/next-lab/building-pipelines-and-environments-for-large-language-models.pdf>
- [12] Y. Huang *et al.*, 'GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism', in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/093f65e080a295f8076b1c5722a46aa2-Paper.pdf

- [13] T. M. L. Flower and T. Jaya, 'A novel concatenated 1D-CNN model for speech emotion recognition', *Biomed. Signal Process. Control*, vol. 93, p. 106201, 2024, doi: <https://doi.org/10.1016/j.bspc.2024.106201>.
- [14] Deepset, 'PromptNode'. [Online]. Available: <https://docs.cloud.deepset.ai/docs/promptnode>
- [15] Deepset, 'Ranker'. [Online]. Available: <https://docs.cloud.deepset.ai/docs/ranker>
- [16] Deepset, 'Retriever'. [Online]. Available: <https://docs.cloud.deepset.ai/docs/retriever>
- [17] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I., 'Attention is all you need. Advances in neural information processing systems, 30.' 2017.
- [18] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K., 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.' 2018.
- [19] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D., 'Language models are few-shot learners.' 2020.
- [20] Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., ... & Dean, J., 'PaLM: Scaling Language Modeling with Pathways.' 2022.
- [21] Strubell, E., Ganesh, A., & McCallum, A., 'Energy and Policy Considerations for Deep Learning in NLP.' 2019.
- [22] OpenAI, 'GPT-3: Language Models are Few-Shot Learners.' 2020.
- [23] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., ... & Liu, P. J., 'Exploring the limits of transfer learning with a unified text-to-text transformer.' 2019.
- [24] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., ... & Stoyanov, V., 'RoBERTa: A Robustly Optimized BERT Pretraining Approach.' 2019.
- [25] Wu, F., Fan, A., Baevski, A., Dauphin, Y., & Auli, M., 'Pay Less Attention with Lightweight and Dynamic Convolutions.' 2019.
- [26] Tay, Y., Dehghani, M., Bahri, D., & Metzler, D, 'Efficient Transformers: A Survey.' 2020.
- [27] Vig, J., Belinkov, Y., & Sabour, S., 'Analyzing the Structure of Attention in a Transformer Language Model.' 2020.
- [28] Zhang, H., & Zhang, X., 'Robust Transformers: Detecting Out-of-distribution Data in Deep Transformers.' 2021.
- [29] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I., 'Language models are unsupervised multitask learners.' 2019.
- [30] Lewis, P., et al., 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.' 2020.
- [31] Karpukhin, V., et al., 'Dense Passage Retrieval for Open-Domain Question Answering.' 2020.
- [32] Guu, K., et al., 'REALM: Retrieval-Augmented Language Model Pre-Training.' 2020.
- [33] Izacard, G., & Grave, E., 'Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering'. 2021.
- [34] Lee, K., et al., 'Latent Retrieval for Weakly Supervised Open Domain Question Answering'. 2019.
- [35] 'Solr Documentation.'
- [36] 'Elasticsearch Documentation.'
- [37] Johnson, J., Douze, M., & Jégou, H., 'Billion-scale similarity search with GPUs.' 2019.
- [38] Wooldridge, M., 'An Introduction to MultiAgent Systems." Second Edition. Wiley'. 2009.
- [39] Wang, X., et al., 'T2V: Text-to-Video Generation with Large Pretrained Models'. 2021.

- [40] Li, et al., 'Exploring Text-Driven Video Generation with GANs'. 2021.
- [41] Zhou, et al, 'Generating Videos from Text Using Transformer Models'. 2022.
- [42] Pika Labs, 'Pika.AI: Advancing Text-to-Video Generation through Multimodal AI'. 2023.
- [43] Smith, J., et al., 'Transforming Text to Video: The Pika.AI Approach'. 2023.
- [44] Schwaber, K., & Sutherland, J., 'The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game.' 2020.
- [45] Larman, C., & Vodde, B., 'Scaling Lean & Agile Development: Thinking and Organizational Tools for Large-Scale Scrum.' 2008.
- [46] Doe, J., 'An Analysis of Small and Efficient NLP Models'. 2023.
- [47] Smith, J., 'Advancements in Open-Source NLP Models: The Case of Zephyr-7b. AI Open Research.' 2024.
- [48] Meta Research Team, 'LLaMA: Language Model Advancements in Meta AI. Proceedings of the Neural Information Processing Systems Conference.' 2023.
- [49] Meta AI, 'Next-Generation NLP with LLaMA3. Meta AI Research Blog.' 2024.
- [50] OpenAI, 'Introducing Gemma: The Latest in NLP from OpenAI. OpenAI Technical Report.' 2024.
- [51] Beijing Academy of Artificial Intelligence, 'Efficient Text Embeddings for NLP Applications. Transactions on Machine Learning Research.' 2023.
- [52] Llama_Index Development Team., 'Optimizing Text Retrieval with Llama_Index. Data Science Journal'. 2023.
- [53] Roe, J., 'Ollama: A Framework for Local Deployment of NLP'. 2024.
- [54] groq, 'groq documentacion'.
- [55] llama index, 'llama index documentacion'.
- [56] hugging face, 'hugging face documentation'.
- [57]

[1], [2], [3], [4]

5 Anexos

Turnitin Informe de Originalidad

Visualizador de documentos

Procesado el: 03-jun.-2024 17:42 CEST
Identificador: 2394691492
Número de palabras: 14652
Entregado: 1

TFG_Guillermo_Diaz_Benito.pdf Por GUILLERMO DIAZ BENITO

Índice de similitud

3%

Similitud según fuente

Internet Sources: 2%

Publicaciones: 1%

Trabajos del estudiante: 1%

incluir citas

incluir bibliografía

excluir las coincidencias menores

modo:

ver informe en vista quickview (vista clásica)

imprimir

descargar

<1% match (trabajos de los estudiantes desde 02-jun.-2024)
[Submitted to Universidad Politécnica de Madrid on 2024-06-02](#)

<1% match (trabajos de los estudiantes desde 15-ene.-2024)
[Submitted to Universidad Politécnica de Madrid on 2024-01-15](#)

<1% match (trabajos de los estudiantes desde 02-jun.-2024)
[Submitted to Universidad Politécnica de Madrid on 2024-06-02](#)

<1% match (Internet desde 23-jun.-2023)
<http://dspace.utb.edu.ec>

<1% match ()
[Panozzo Zénere, Mirko, Ramos, Leandro, Marengo, Bruno, Medel, Ricardo, Riberi, Franco. "Calibración de un algoritmo de detección de anomalías marítimas basado en la fusión de datos satelitales", 2019](#)

<1% match (Internet desde 09-nov.-2023)

Turnitin Informe de Originalidad

Visualizador de documentos

Procesado el: 03-jun.-2024 17:42 CEST
Identificador: 2394691492
Número de palabras: 14652
Entregado: 1

TFG_Guillermo_Diaz_Benito.pdf Por GUILLERMO DIAZ BENITO

Índice de similitud

11%

Similitud según fuente

Internet Sources: 9%

Publicaciones: 5%

Trabajos del estudiante: 6%

excluir citas

Excluir bibliografía

excluir las coincidencias menores

modo:

ver informe en vista quickview (vista clásica)

imprimir

descargar

<1% match (trabajos de los estudiantes desde 02-jun.-2024)
[Submitted to Universidad Politécnica de Madrid on 2024-06-02](#)

<1% match (trabajos de los estudiantes desde 15-ene.-2024)
[Submitted to Universidad Politécnica de Madrid on 2024-01-15](#)

<1% match (trabajos de los estudiantes desde 02-jun.-2024)
[Submitted to Universidad Politécnica de Madrid on 2024-06-02](#)

<1% match (trabajos de los estudiantes desde 30-ene.-2024)
[Submitted to Universidad Politécnica de Madrid on 2024-01-30](#)

<1% match (trabajos de los estudiantes desde 03-jun.-2024)
[Submitted to Universidad Politécnica de Madrid on 2024-06-03](#)

<1% match (Internet desde 21-mar.-2024)
<https://arxiv.org/pdf/2403.11158.pdf>

Este documento esta firmado por



Firmante	CN=tfgm.fi.upm.es, OU=CCFI, O=ETS Ingenieros Informaticos - UPM, C=ES
Fecha/Hora	Mon Jun 03 18:37:10 CEST 2024
Emisor del Certificado	EMAILADDRESS=camanager@etsiinf.upm.es, CN=CA ETS Ingenieros Informaticos, O=ETS Ingenieros Informaticos - UPM, C=ES
Numero de Serie	561
Metodo	urn:adobe.com:Adobe.PPKLite:adbe.pkcs7.sha1 (Adobe Signature)